

UNIDAD 8.

DESARROLLO DE INTERFACES GRÁFICAS

**Programación
CFGS DAW**

Adaptado por: José Miguel Blázquez
Autores: Carlos Cacho, Raquel Torres y Lionel Tarazón

2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
2. Creación de una interfaz gráfica Swing con NetBeans.....	5
2.1 Crear un proyecto y la ventana de la aplicación.....	6
2.2 Añadir un contenedor JPanel.....	9
2.3 Añadir componentes al contenedor.....	11
3. Jerarquía de componentes.....	14
4. JFrame.....	17
4.1 Crear y configurar un <i>JFrame</i>	18
4.2 Getters y Setters de los objetos auxiliares.....	18
4.3 Visibilidad y tamaño.....	18
5. Eventos.....	19
5.1 Cómo gestionar un evento.....	19
5.2 Ejemplo.....	21
6. Componentes.....	22
6.1 Superclase JComponent.....	22
6.2 Etiquetas (JLabel).....	23
6.3 Botones.....	24
6.4 Campos de texto.....	28
6.5 Rangos.....	33
6.6 Listas y cajas.....	34
6.7 Menús JMenuBar, JMenu y JMenuItem.....	36
6.8 Panel con pestañas JTabbedPane.....	41
6.9 Selector de ficheros JFileChooser.....	42
6.10 Tablas: el componente JTable.....	45
7. Cuadros de diálogo: JOptionPane.....	47
7.1 showMessageDialog.....	47
7.2 showConfirmDialog.....	48
7.3 showInputDialog.....	48
7.4 showOptionDialog.....	49
8. Layouts: control de distribución de componentes.....	50
9. Ejemplo guiado de interfaz simple.....	52
10. Patrones de diseño y arquitecturas software.....	55
10.1 MVC (Modelo, Vista, Controlador).....	56
10.2 Singleton.....	59
10.3 Observer.....	60
10.4 Factory.....	62

UD8. DESARROLLO DE INTERFACES GRÁFICAS

1. INTRODUCCIÓN

El objetivo de esta unidad es aprender a diseñar pequeñas interfaces gráficas empleando para ello la librería gráfica *Swing*. Si bien existen otras librerías más modernas, como JavaFX, la elección de Swing se debe a 2 motivos: sólo necesitamos adquirir fundamentos de programación de interfaces y, por otro lado, Swing está muy consolidado y hay abundante material para investigar.

Una interfaz gráfica de usuario, conocida también como GUI (del inglés *graphical user interface*), es un elemento de un programa informático que actúa de frontera entre el software y el usuario utilizando imágenes y objetos gráficos para representar la información y las acciones disponibles. Su principal uso consiste en proporcionar un entorno visual sencillo para permitir la comunicación e interacción del usuario con el programa. Por ejemplo en un navegador de Internet, un editor de textos, una aplicación de móvil, etc. Todos tienen ventanas, texto, imágenes, botones, menús, barras, etc. que nos permiten ver la información representada por el programa e interactuar con el para realizar acciones como visitar una página web, escribir un texto con formato, guardar o abrir archivos del disco duro, escuchar música, etc.

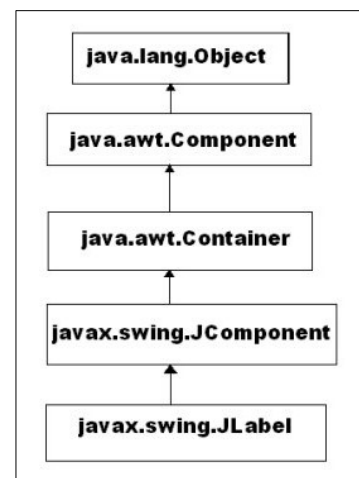
En Java la programación de aplicaciones con interfaces gráficas se realiza utilizando diversas librerías; en esta unidad estudiaremos **Swing**, una biblioteca gráfica que incluye widgets para interfaz gráfica de usuario tales como cajas de texto, botones, listas desplegables y tablas. Swing está basado en la Abstract Window Toolkit (AWT, en español Kit de Herramientas de Ventana Abstracta), que es un kit de herramientas de gráficos, interfaz de usuario y sistema de ventanas independiente de la plataforma original de Java. La programación de interfaces gráficas es más compleja que las interfaces por consola, en las que solo es necesario programar la entrada y salida por teclado, pero también permite una interacción más rica y versátil, dando como resultado una experiencia de usuario más sencilla y amigable.

Debido a la complejidad del desarrollo de interfaces gráficas, muchos Entornos de Desarrollo, como NetBeans, incorporan herramientas que permiten diseñar aplicaciones gráficas de forma sencilla.

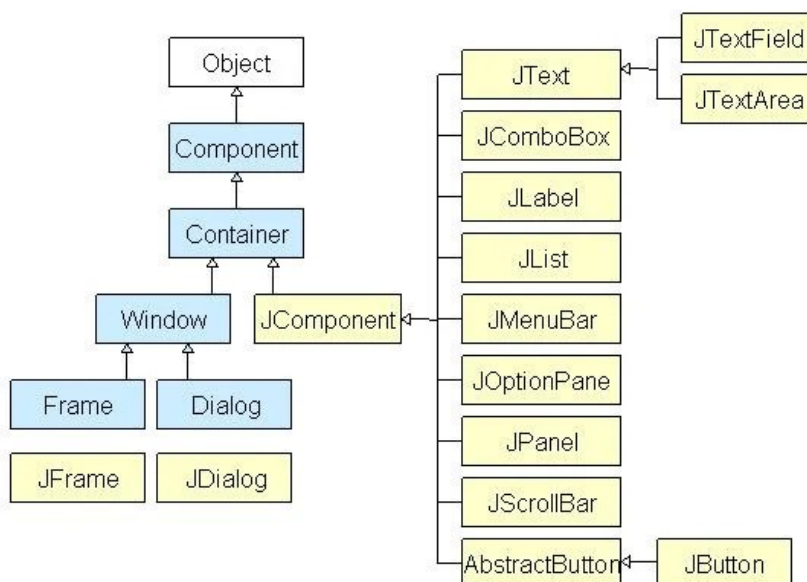
2. CREACIÓN DE UNA INTERFAZ GRÁFICA SWING CON NETBEANS

Todos los elementos *swing* heredan de la clase *javax.swing.JComponent*, la cual hereda de *java.awt.Container* y ésta, a su vez, de *java.awt.Component*. Conocer esta jerarquía nos permitirá entender algunas de las características comunes.

Por ejemplo, a la derecha podemos ver la jerarquía de herencia del componente *JLabel*, que representa una etiqueta en la interfaz gráfica.



Veamos la jerarquía de elementos Swing un poco más ampliada:



¿Eso es todo? No, se trata de un resumen. La jerarquía de elementos completa es mucho más extensa, pero poco a poco vamos a ir ampliándola.

Como primer ejemplo **vamos a crear paso a paso una interfaz gráfica muy sencilla** que contendrá una ventana con tres componentes:

- Una etiqueta (*JLabel*).
- Un campo de texto (*JTextField*).
- Un botón (*JButton*).

Los pasos que vamos a seguir son los siguientes:

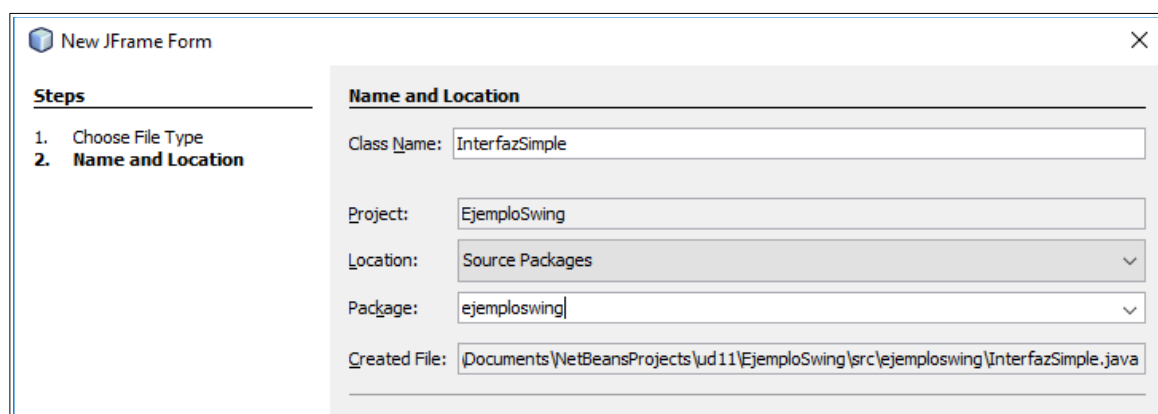
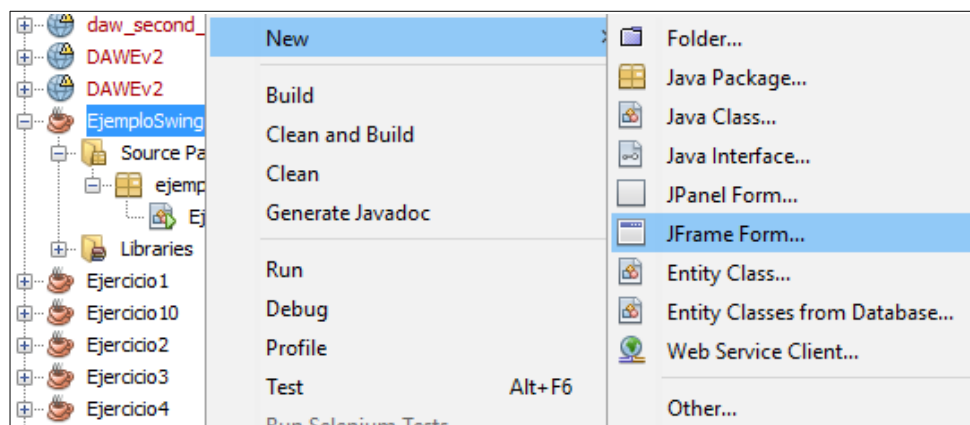
1. Crear un proyecto
2. Crear una ventana de aplicación.
3. Añadir un contenedor.
4. Añadir componentes al contenedor.
5. Cambiar propiedades de los componentes.

2.1 Crear un proyecto y la ventana de la aplicación

La creación del proyecto es igual a la que hemos visto hasta ahora. Le daremos el nombre de “EjemploSwing” y comprobaremos que esté **desmarcada** la casilla de “Create Main Class”. Para crear una ventana de aplicación hay que crear un componente **JFrame** (*javax.swing.JFrame*).

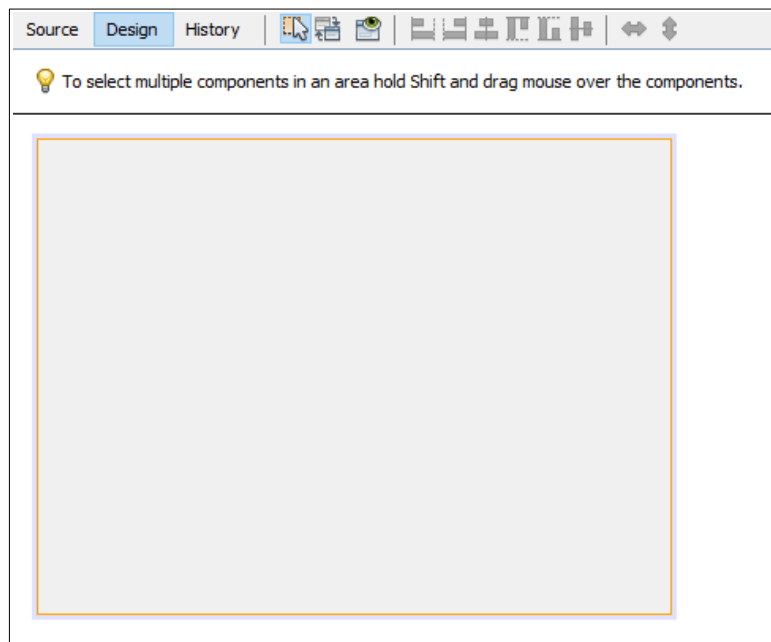
eleccionaremos el proyecto *EjemploSwing* que hemos creado y con el botón derecho del ratón seleccionamos la opción *New > JFrame Form*.

- Escribimos *InterfazSimple* como nombre de la clase.
- Añadimos *EjemploSwing* como nombre del *package* (paquete).
- Pulsamos *Finish*.

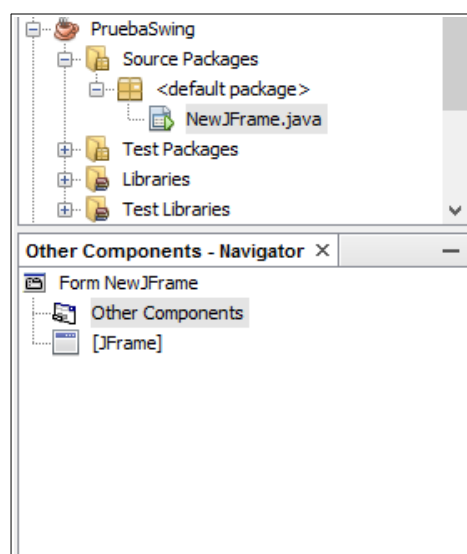


Ahora tendremos una clase *InterfazSimple.java* en modo diseño para construir nuestra aplicación. En este punto tenemos nuestro proyecto creado para iniciar nuestra aplicación. En el entorno NetBeans podemos distinguir:

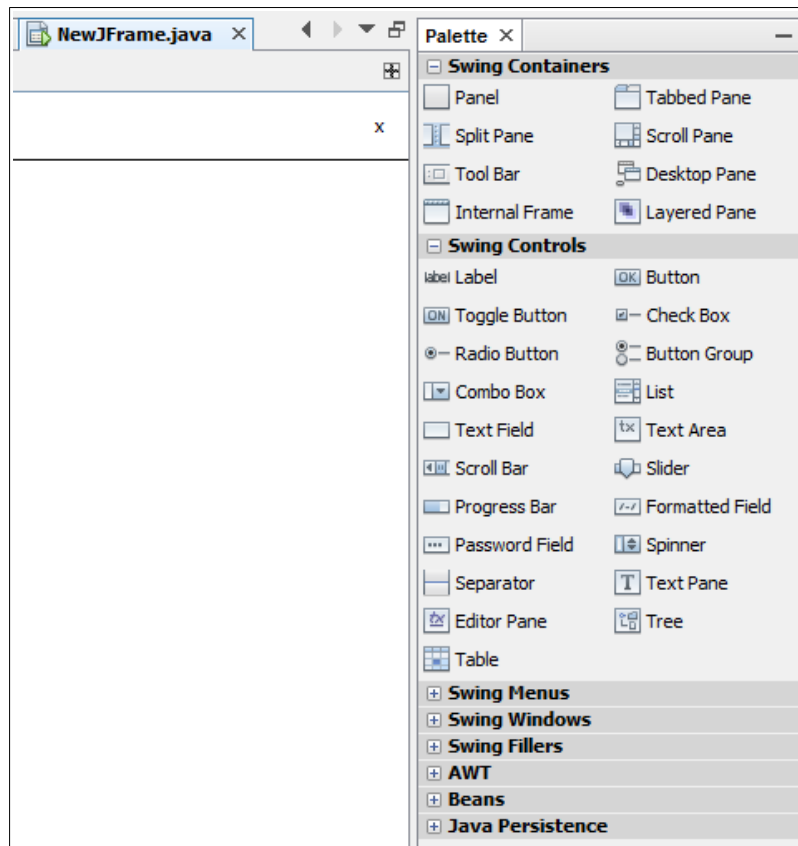
- **Design Area:** Ventana principal para crear y modificar la GUI (Graphic User Interface).



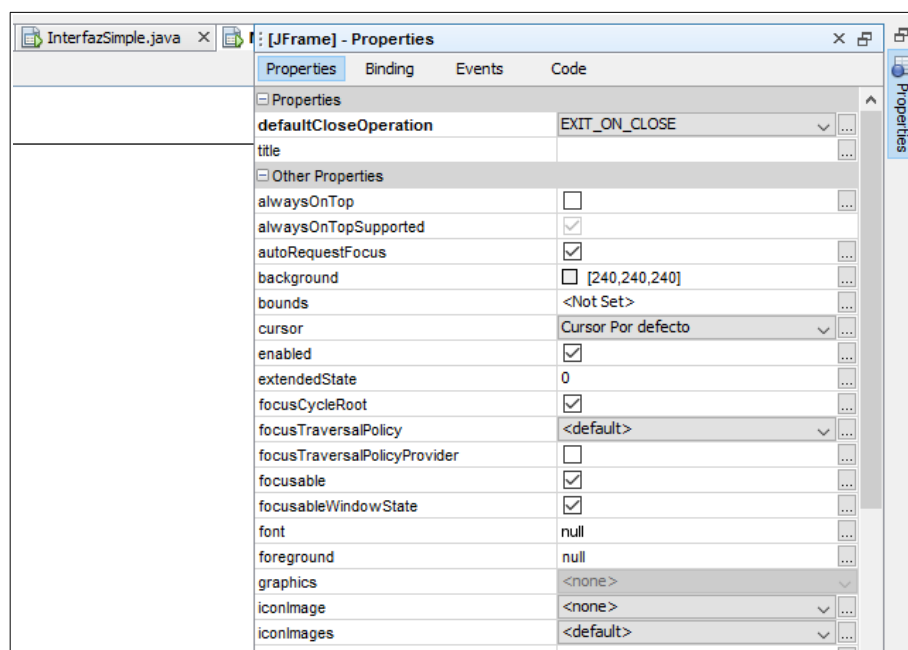
- **Navigator:** Muestra la jerarquía de componentes de nuestra aplicación.



- **Palette:** Lista de todos los componentes que podemos añadir a nuestra aplicación gráfica.



- **Properties:** Muestra las propiedades del componente seleccionado.

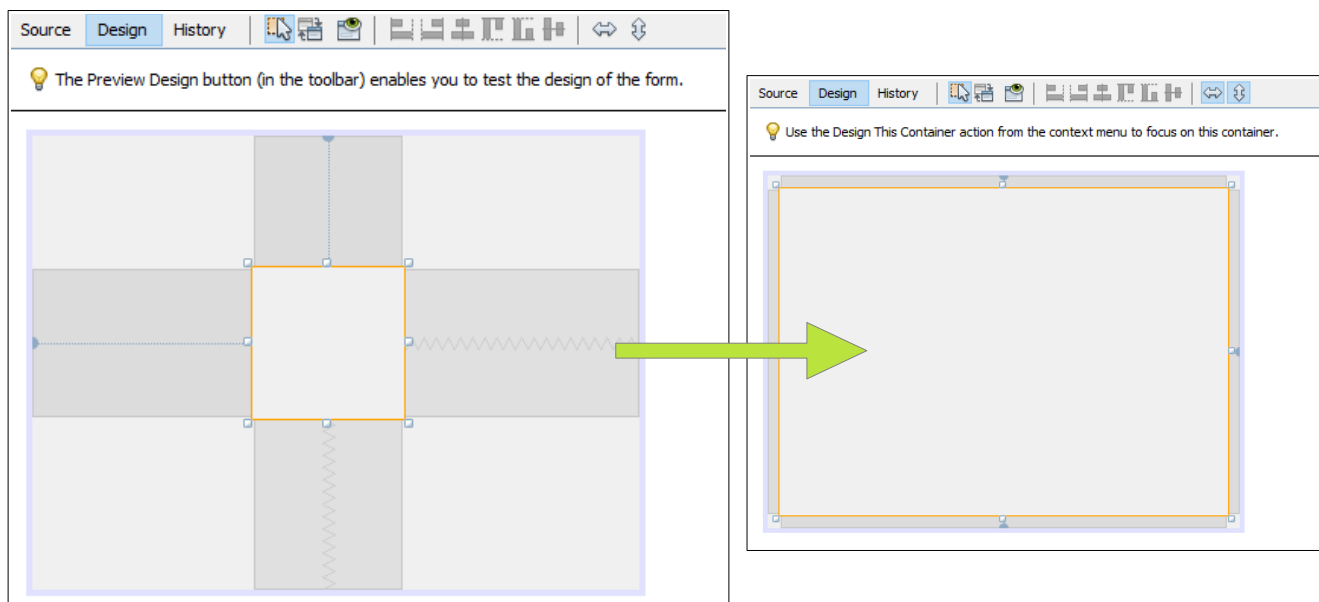


2.2 Añadir un contenedor JPanel

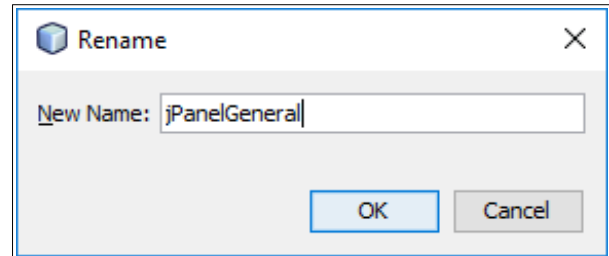
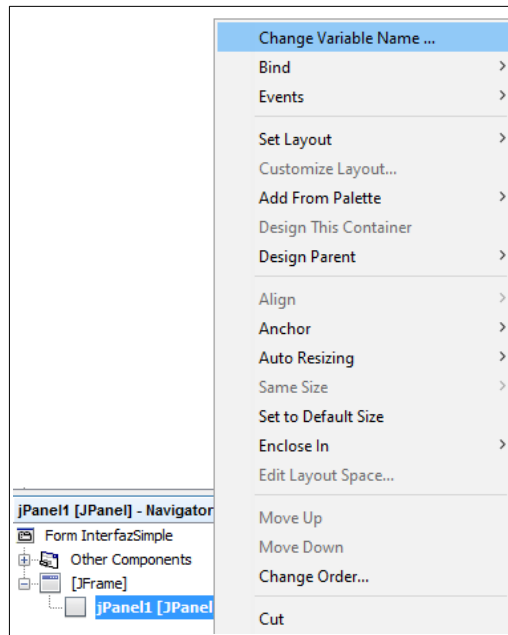
Un contenedor es un elemento no visual para organizar los componentes de nuestra aplicación, es decir: los botones, texto, menús, imágenes, etc.

En *Swing* existen muchos tipos de contenedores. Sus diferencias radican en la forma en la que manejan los componentes que tienen dentro. Por ejemplo, el *JTabbedPane* (*javax.swing.JTabbedPane*) es un contenedor con pestañas donde cada una está asociada a un componente. El *JSplitPane* es un contenedor que se comporta como un panel dividido en dos.

Nosotros utilizaremos el **contenedor** de propósito general **JPanel** (*javax.swing.JPanel*), que es el más sencillo de todos y puede contener cualquier número de componentes en su interior. La posición y tamaño de los componentes es configurable. Pinchamos sobre *Panel* en *Palette* y sin soltar, lo arrastramos dentro del contenedor *JFrame* en el área de *Diseño*. Al hacerlo veremos una pantalla tal como esta; desde las esquinas lo ajustamos a los bordes del *Jframe*, dejando unos pequeños márgenes.



Luego, en el panel *Navigator* seleccionamos *jPanel1*, hacemos clic-derecho y marcamos la opción "Change Variable Name..." para cambiarle el nombre a *jPanelGeneral*. Es **MUY recomendable** renombrar contenedores y componentes para aclararnos mejor en el proceso de desarrollo.



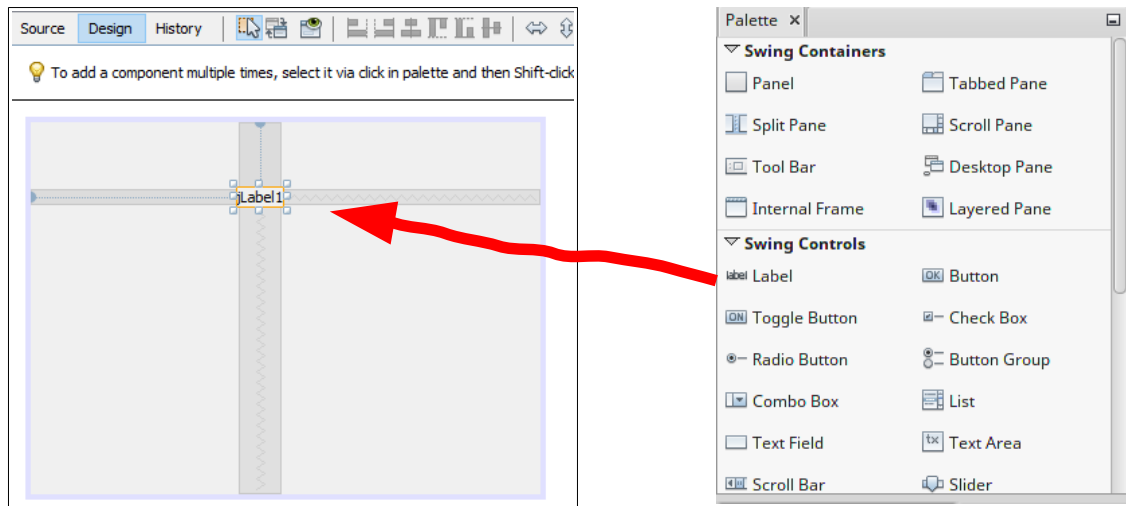
Ahora sólo queda añadir los 3 componentes:

1. Una etiqueta (*JLabel*).
2. Un campo de texto (*TextField*).
3. Un botón (*Button*).

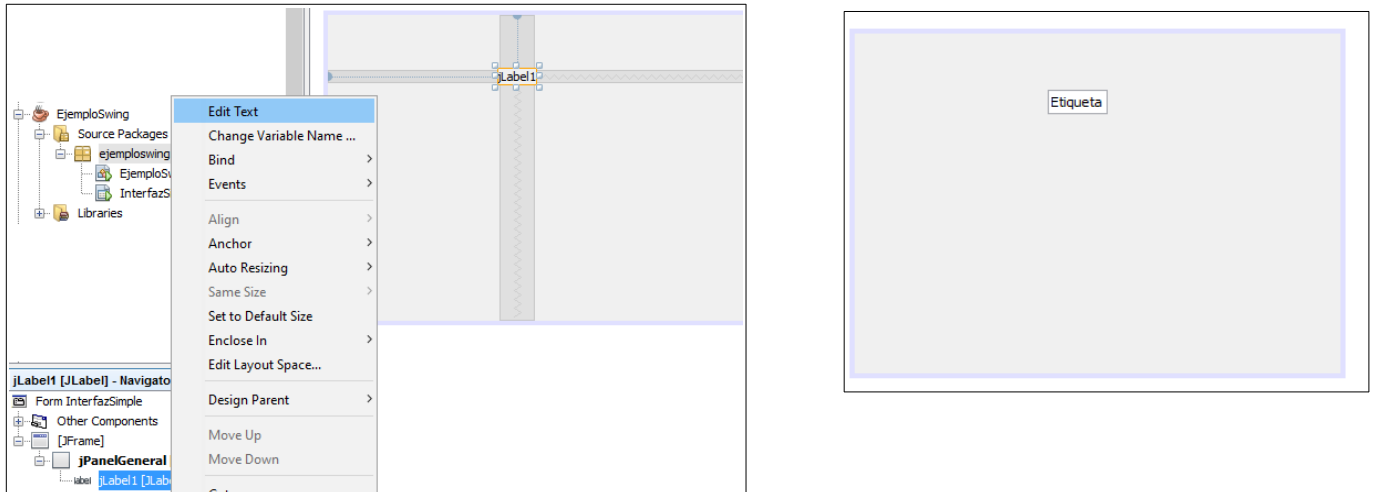
2.3 Añadir componentes al contenedor

1. Añadir un componente etiqueta (JLabel):

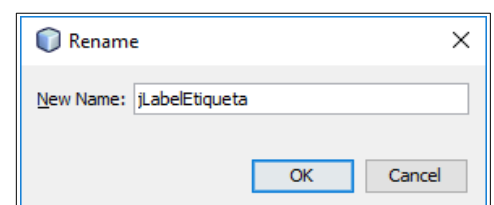
- Seleccionamos el componente *Label* (*javax.swing.JLabel*) del panel *Palette*.
- Lo arrastramos y soltamos dentro del *JPanelGeneral* que hemos creado antes.



- Seleccionamos del panel *Navigator* el nuevo componente creado *jLabel1*.
- Con el botón derecho del ratón -> *Edit text* y escribimos *Etiqueta*

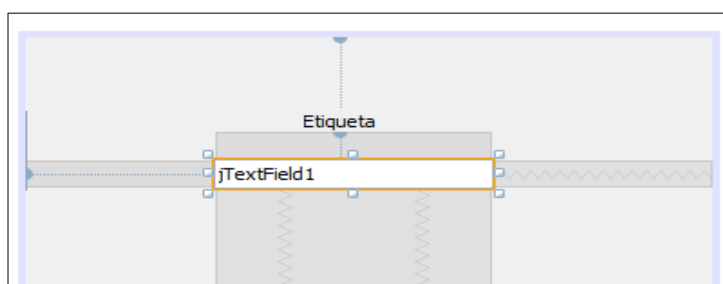


- Y de nuevo con el botón derecho del ratón -> *Change Variable Name ...* -> *jLabelEtiqueta* (recuerda que cambiar el nombre no es obligatorio pero sí recomendable). OJO: el texto de la etiqueta y el nombre del objeto no son lo mismo.



2. Añadir un componente cuadro de texto (JTextField):

- Seleccionamos el componente *JTextField* (*javax.swing.JTextField*) del panel *Palette*.
- Lo añadimos dentro del *JPanelGeneral* que hemos creado previamente.
- Una vez añadido y posicionado en el área deseada, seleccionamos del panel *Navigator* el nuevo componente creado *jTextField1*.
- Con el botón derecho del ratón -> *Change Variable Name ...*-> *jTextFieldTexto*

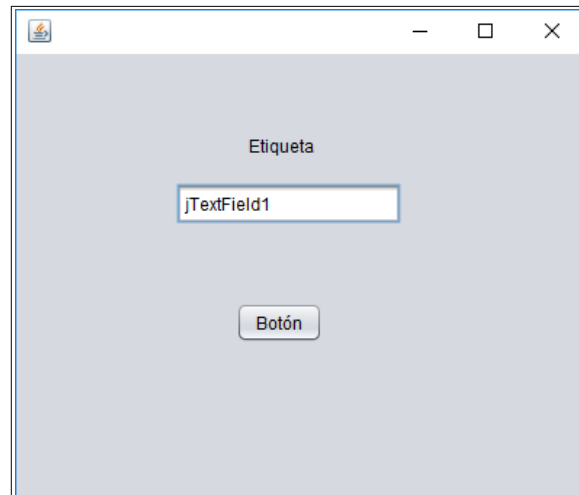


3. Añadir un componente botón (JButton):

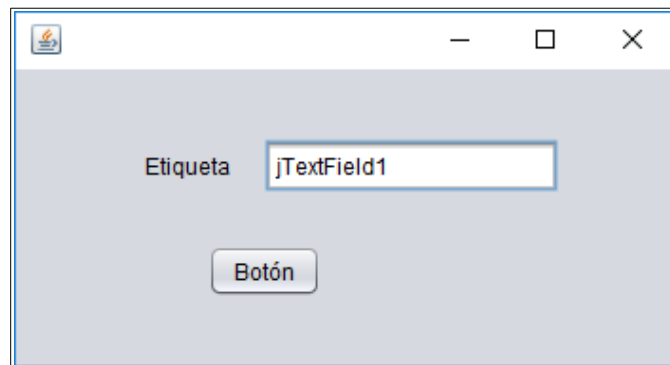
- Seleccionamos el componente *JButton* (*javax.swing.JButton*) del panel *Palette*.
- Lo añadimos dentro del *JPanelGeneral* que hemos creado previamente.
- Una vez añadido y posicionado en el área deseada, seleccionamos del panel *Navigator* el nuevo componente creado *jButton1*.
- Con el botón derecho del ratón -> *Edit text* y escribimos "Botón".
- Con el botón derecho del ratón -> *Change Variable Name ...*-> *jButtonBoton*.



Ya podemos ver cómo queda el aspecto de nuestra primera aplicación *Swing* con *NetBeans*. Ejecutamos con F6 o con el icono de ejecutar.

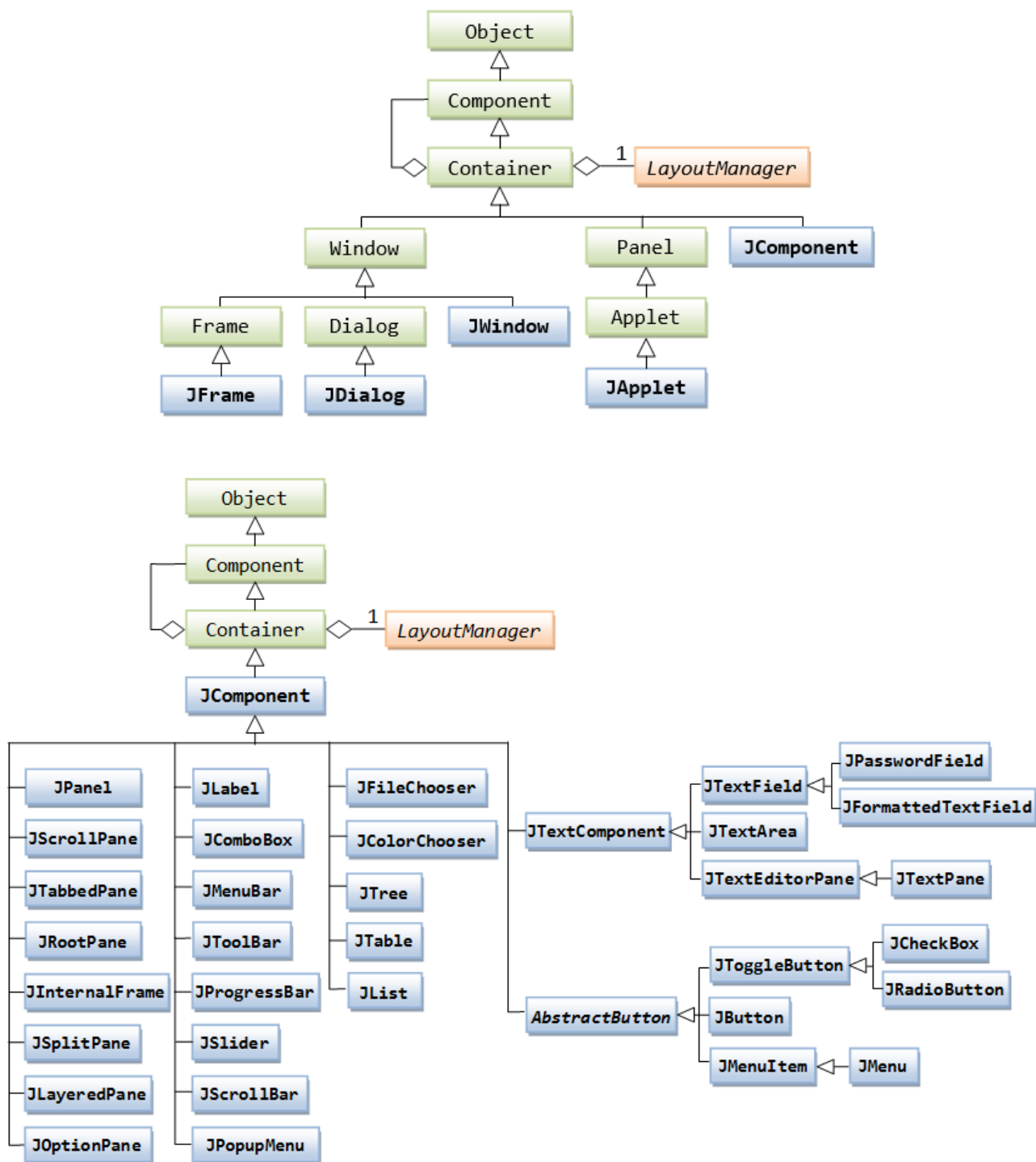


Desde el modo de diseño podemos mover los diferentes elementos para que queden así:



3. JERARQUÍA DE COMPONENTES

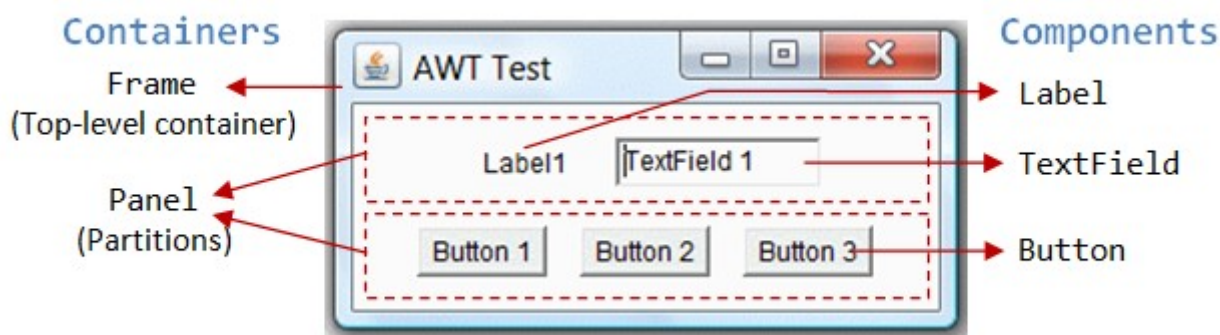
Como la mayoría de clases de Java, los componentes utilizados en el AWT y Swing pertenecen a una determinada jerarquía. Esta jerarquía de clases (que vimos parcialmente al empezar el tema) se muestra en la siguiente figura. Todos los componentes descienden de la clase *Component*, de la que ya heredan algunos métodos interesantes. Los elementos pertenecientes a AWT van en **verde** y los elementos de Swing en **azul**.



Características fundamentales de la jeraquía de Componentes de AWT y Swing:

1. Todos los **Components** (excepto *Window* y los que derivan de ella) deben ser añadidos a un **Container**. También un **Container** puede ser añadido a otro **Container**.
2. Para añadir un **Component** a un **Container** se utiliza el método `add()` de la clase **Container**: `containerName.add(componentName)`.
3. Los **Containers** de máximo nivel son las **Windows (Frames y Dialogs)**. Los **Panels** y **ScrollPanels** deben estar siempre dentro de otro **Container**.
4. Un **Component** sólo puede estar en un **Container**. Si está en un **Container** y se añade a otro, deja de estar en el primero.
5. La clase **Component** tiene una serie de funcionalidades básicas comunes (atributos y métodos) que son heredadas por todas sus subclases.

Finalmente, es importante recordarte que existen 2 “grupos” de elementos para el desarrollo de interfaces con Swing: los componentes y los contenedores.

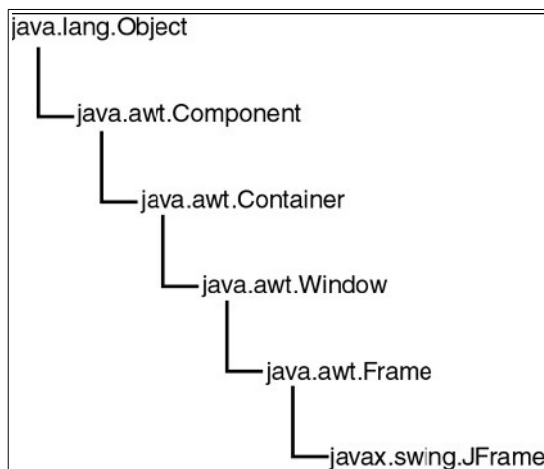


En la figura anterior, hay 3 contenedores: un Frame y dos Paneles. Un Frame es el contenedor de nivel superior de un programa. Un frame tiene una barra de título (que contiene un icono, un título y los botones minimizar/maximizar/cerrar), una barra de menú opcional y el área de visualización de contenido. Un Panel es un área rectangular que se utiliza para agrupar componentes GUI relacionados en un diseño. El frame de nivel superior contiene 2 paneles. Hay 5 componentes: 1 etiqueta (label: que proporciona una descripción), 1 campo de texto (para que los usuarios ingresen texto) y 3 botones (para que el usuario active ciertas acciones programadas).

4. JFRAME

De todos los contenedores existentes, nos vamos a centrar en el `JFrame`. Es un tipo de **ventana y contenedor sencillo** que emplearemos para situar en él todos los demás componentes que necesitemos para el desarrollo de la interfaz de nuestro programa: la ventana principal.

En el gráfico se muestra la jerarquía de herencia de este componente desde *Object*, que es el padre de todas las clases de Java. Los métodos de este componente estarán repartidos a lo largo de todos sus ascendientes. Así, por ejemplo, resulta intuitivo que debiera haber un método para cambiar el color de fondo del *frame*, pero él no tiene ningún método para ello, lo tiene *Component*.



Para ver el código que hay detrás para la creación de un *JFrame* podemos pasar de la pestaña **Design** a la pestaña **Source** de *Netbeans*. El código lo genera *NetBeans* automáticamente a medida que añadimos contenedores y componentes. Veamos brevemente algunos de sus elementos a título informativo:

```
10  /**
11  public class JFramePg17 extends javax.swing.JFrame {
12
13      /**
14       * Creates new form JFramePg17
15       */
16      public JFramePg17() {
17          initComponents();
18      }
19
20      /**
21       * This method is called from within the constructor to initialize the form.
22       * WARNING: Do NOT modify this code. The content of this method is always
23       * regenerated by the Form Editor.
24       */
25      @SuppressWarnings("unchecked")
26      Generated Code
27
28      /**
29       * @param args the command line arguments
30       */
31      public static void main(String args[]) {
32          /* Set the Nimbus look and feel */
33          Look and feel setting code (optional)
34
35          /* Create and display the form */
36          java.awt.EventQueue.invokeLater(new Runnable() {
37              public void run() {
38                  new JFramePg17().setVisible(true);
39              }
40          });
41      }
42
43      // Variables declaration - do not modify
44      // End of variables declaration
45  }
```


Veamos los métodos más utilizados del *JFrame*.

4.1 Crear y configurar un *JFrame*

Método	Propósito
<code>JFrame()</code> y <code>JFrame(String)</code>	Crean un <i>frame</i> , opcionalmente se pasa el título como <i>String</i> .
<code>String getTitle()</code> y <code>setTitle(String)</code>	Getter y setter del título de la ventana.
<code>void setDefaultCloseOperation(int)</code> <code>int getDefaultCloseOperation()</code>	Define o devuelve el tipo de operación que ocurre cuando el usuario pulsa el botón de cerrar la ventana. Toma como valor un número entero definido en la clase <code>WindowConstants</code> . Las posibles opciones son: • <code>WindowConstants.DO_NOTHING_ON_CLOSE</code> • <code>WindowConstants.HIDE_ON_CLOSE</code> (defecto) • <code>WindowConstants.DISPOSE_ON_CLOSE</code>

4.2 Getters y Setters de los objetos auxiliares

Método	Propósito
<code>void setContentPane(Container)</code> <code>Container getContentPane()</code>	Define o devuelve el panel de contenido del <i>frame</i> .
<code>JRootPane createRootPane()</code> <code>void setRootPane(JRootPane)</code> <code>JRootPane getRootPane()</code>	Crea, define o devuelve el panel raíz del <i>frame</i> . El panel raíz maneja el interior de <i>frame</i> , incluyendo el panel de contenido, el panel transparente, etc.
<code>void setJMenuBar(JMenuBar)</code> <code>JMenuBar getJMenuBar()</code>	Define o devuelve la barra de menú del <i>frame</i> .
<code>void setGlassPane(Component)</code> <code>Component getGlassPane()</code>	Define o devuelve el panel transparente del <i>frame</i> .
<code>Void setLayeredPane(JLayeredPane)</code> <code>JLayeredPane getLayeredPane()</code>	Define o devuelve el panel de capas del <i>jframe</i> .

4.3 Visibilidad y tamaño

Algunos métodos útiles heredados de la superclase *Window*.

- **setVisible(boolean)**: Hace visible o invisible la ventana.
- **toFront()**: Si la ventana está visible, la pone delante de las otras ventanas del sistema.
- **toBack()**: Si la ventana está visible, la pone detrás de las otras ventanas del sistema.
- **pack()**: Redimensiona la ventana al tamaño apropiado para mostrar sus componentes,
- **setSize(Dimension)**: Establece el tamaño de la ventana a partir de un objeto *Dimension*.

5. EVENTOS

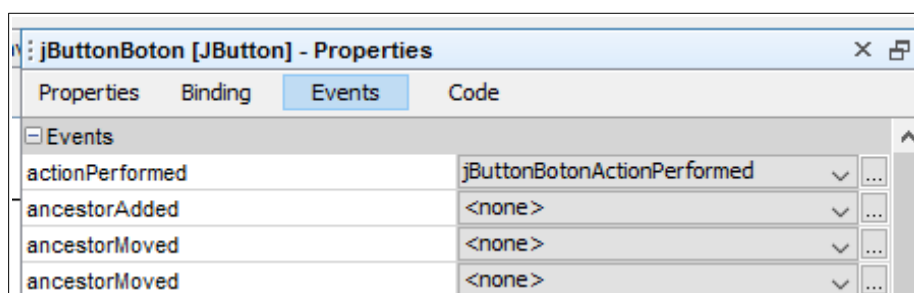
Todos los sistemas operativos están constantemente atendiendo a los eventos generados por los usuarios. Estos eventos pueden ser: **pulsar una tecla del teclado, mover el ratón, hacer clic izquierdo, clic derecho, mover la rueda del ratón, etc.** (Java distingue entre simplemente pulsar el ratón en un sitio cualquiera o hacerlo, por ejemplo, en un botón). Cuando se produce un evento el sistema operativo notifica a la aplicación involucrada y ellas deciden qué hacer con dicho evento (se ejecuta un método asociado al evento).

Al contrario que en los programas que hemos desarrollado hasta ahora, los programas con interfaz gráfica seguirán un flujo acotado y guiado por los eventos que ocurran, generalmente ocasionados voluntariamente con las acciones del usuario, por ejemplo, cuando se pulse un botón. Esto es lo que llamamos **programación orientada a eventos (también reactiva o asíncrona)**.

El modelo de Java se basa en la **delegación de eventos**: el evento se produce en un determinado componente, por ejemplo un `JButton`, que será la “fuente del evento” (`ActionListener`). A continuación el evento se transmite a un “manejador de eventos” (`EventListener`) que esté asignado al componente en el que se produjo el evento. El objeto que escucha los eventos es el que se encargará de responder a ellos adecuadamente. Esta separación de código entre generación del evento y actuación respecto a él simplifica el código y facilita la labor del programador.

5.1 Cómo gestionar un evento

Desde la pestaña de diseño de NetBeans, junto a las propiedades del componente seleccionado, está la **opción “Events” en la aparecen todos los diferentes eventos que se pueden producir en el componente seleccionado.**



Crear un manejador de eventos genérico (evento *actionPerformed*) es muy sencillo. Solo hay que hacer doble-clic sobre el componente, y NetBeans hará 2 cosas en el código:

1. Creará un método para manejar (responder) el evento.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

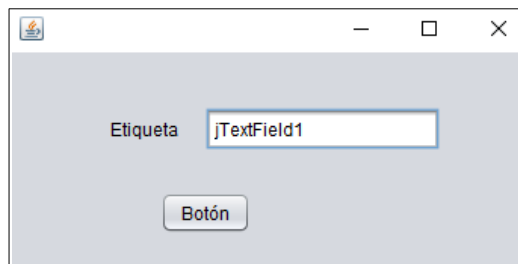
2. Añadirá al componente un ActionListener asociado al método anterior.

```
jButton1.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        jButton1ActionPerformed(evt);  
    }  
});
```

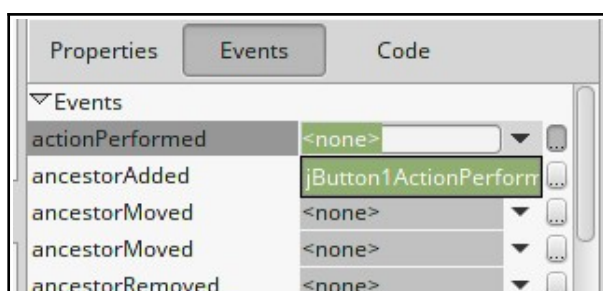
Este código hará que cuando se presione el jButton1 se ejecutará el método jButton1ActionPerformed. **En ese método podremos escribir el código Java que queramos.** Por ejemplo hacer un cálculo matemático y mostrarlo, buscar en un array, leer de un archivo, abrir otra ventana, conectar con una base de datos, etc.

5.2 Ejemplo

Vamos a configurar un evento en el *JFrame* creado en el apartado 2. Queremos que cuando se presione el botón aparezca un mensaje con el texto que el usuario haya introducido en el campo de texto. En este momento no es importante el código del manejador, **lo importante es ver cómo asociar el código al evento del botón con NetBeans**.



1. Desde la pestaña de diseño seleccionamos el botón.
2. Hacemos doble clic con el componente seleccionado o bien en la pestaña *Events* pinchamos el desplegable junto a *actionPerformed* y creamos el manejador.



3. En el método manejador, introducimos el siguiente código:

JOptionPane.showMessageDialog(this, jTextFieldTexto.getText());

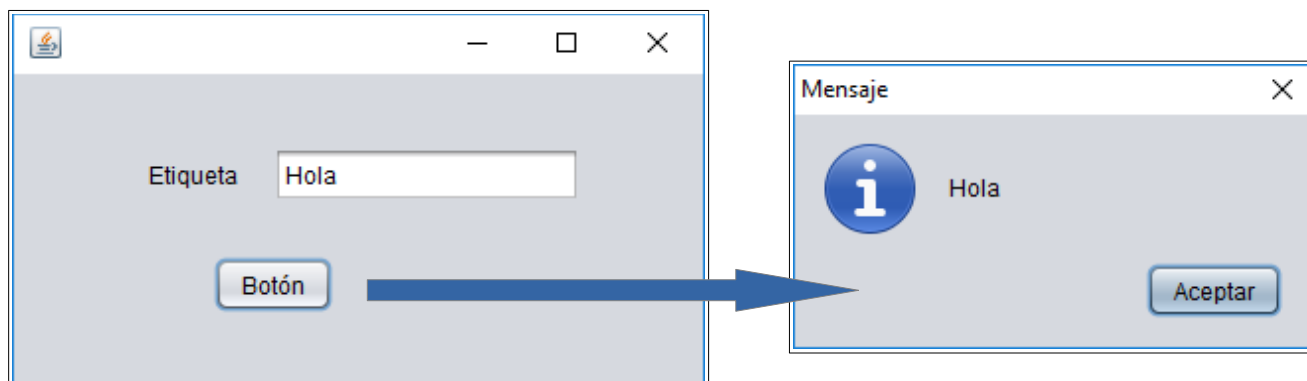
```

97 |
98 | private void jButtonBotonActionPerformed(java.awt.event.ActionEvent evt) {
99 |     // TODO add your handling code here:
100 |     JOptionPane.showMessageDialog(this, jTextFieldTexto.getText());
101 | }

```

NOTA: Será necesario añadir al archivo un `import javax.swing.JOptionPane;`

Con *JOptionPane.showMessageDialog(...)* se mostrará un cuadro de diálogo (una ventanita) con el texto que se le pase como segundo argumento. En este caso, el argumento es el texto de *jTextFieldTexto*, que obtenemos llamando al método *getText()* de dicho objeto.



6. COMPONENTES

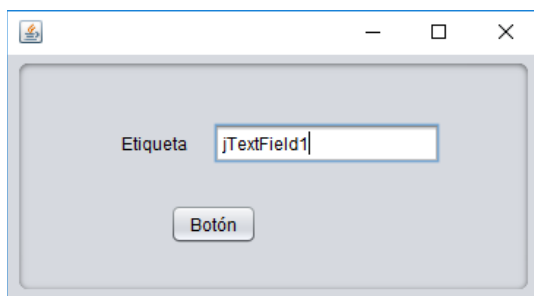
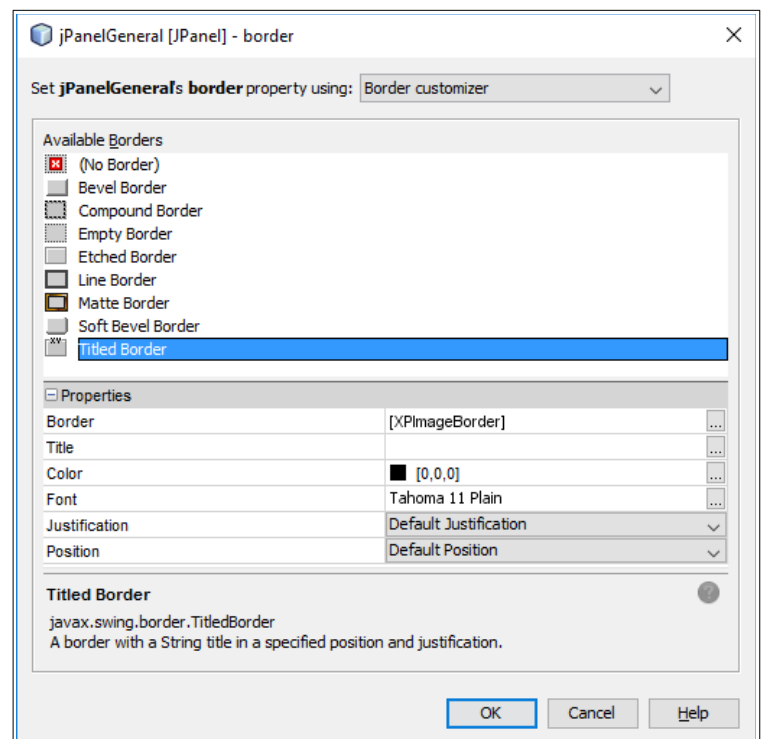
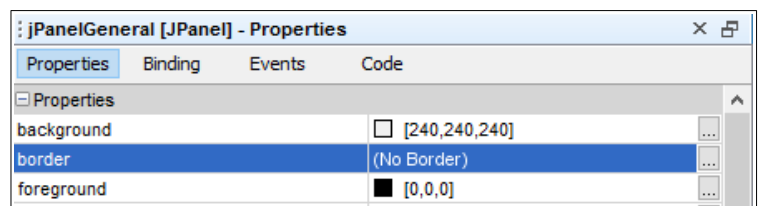
6.1 Superclase JComponent

La clase *JComponent* es la superclase de la que heredan todos los contenedores, ventanas, frames y componentes Swing. Esta clase contiene muchos métodos útiles para configurar el aspecto y comportamiento de los componentes.

No es necesario conocer todos sus métodos pero, a modo de ejemplo, vamos a ver cómo se utiliza el *setBorder()* para configurar el borde de un componente visible. Partiendo de nuestra aplicación gráfica de ejemplo, vamos a configurar el panel de la ventana con un borde de tipo *TitledBorder*.

Pasos a seguir:

1. Utilizando el panel *Navegador* selecciona el *JPanelGeneral*.
2. En la pestaña *Properties* pulsa *border*.
3. Selecciona *TitledBorder* y dale a *OK*.
4. Puedes modificar también otras opciones (título, color, fuente...).
5. Guarda los cambios y ejecuta la aplicación para ver el resultado.



Prueba distintas opciones para ver el resultado.

6.2 Etiquetas (JLabel)

Las etiquetas, junto con los botones y las cajas de selección, son uno de los componentes más básicos de toda interfaz de usuario. El más simple de todos ellos es la etiqueta, que se limita a **mostrar texto no editable**. En *Netbeans* tan solo hay que seleccionar la etiqueta en el panel *Navigator* y modificar sus propiedades en *Properties*.

Constructores:

```
JLabel()           //etiqueta
JLabel(String)      //etiqueta con texto
JLabel(Icon)        //etiqueta con icono
JLabel(String, int)  //texto y alineación horizontal
JLabel(Icon icon, int) //icono y alineación horizontal
```

Métodos:

```
void setText(String) //establece el texto de la etiqueta
String getText()     //devuelve el texto de la etiqueta
void setIcon(Icon)   //establece el icono de la etiqueta
void setHorizontalAlignment(int) // establece alineación horizontal
```

La **alineación horizontal** se establece con valores int almacenados como variables estáticas en la clase JLabel: *JLabel.LEFT*, *JLabel.CENTER* y *JLabel.RIGHT*. Para **instanciar iconos** puede utilizarse el constructor *new ImageIcon("image.ico")*

6.3 Botones

Swing añade varios tipos de botones y cambia la organización de la selección de componentes: todos los botones, cajas de selección, botones de selección y cualquier opción de un menú deben derivar de *AbstractButton*.

6.3.1 JButton

Botón estándar comúnmente utilizado en las aplicaciones gráficas.

Constructores habituales:

```
JButton()           // botón
JButton(String)      // botón con texto
JButton(Icon)        // botón con icono
```

Métodos:

```
String getText()      // devuelve el texto del botón
void setText(String)   // establece el texto del botón
void setIcon(Icon)     // establece el icono del botón
```

6.3.2 JToggleButton

Botón con estado activado y desactivado.

Constructores:

```
JToggleButton()      // botón
JToggleButton(String) // botón con texto
JToggleButton(Icon)   // botón con icono
```

Métodos:

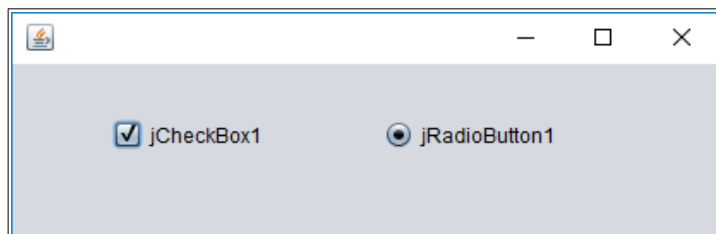
```
boolean getSelected() // devuelve true si está activado,
                      // false si no lo está
void setSelected(boolean) // establece si está activado o no
```

Botón sin pulsar (izquierda) y pulsado (derecha):



6.3.3 JCheckBox y JRadioButton

Componentes tipo *check-box* y *radio*. Es habitual utilizarlos en aplicaciones gráficas para activar o desactivar opciones, así como para elegir una opción de entre varias.



Constructores:

```
JCheckBox(String [, boolean])      //texto y estado (opcional)
JCheckBox(Icon [, boolean])        //icono y estado (opcional)
JCheckBox(String, Icon [, boolean]) //texto, icono y estado
                                   (opcional)

JRadioButton(String [, boolean])   // texto y estado (opcional)
JRadioButton(Icon [, boolean])     // icono y estado (opcional)
JRadioButton(String, Icon [, boolean]) // texto, icono y estado
                                   (opcional)
```

Métodos:

```
void setText(String)    // establece el texto
void setIcon(Icon)      // establece el icono
boolean isSelected()    // devuelve true si activado, false si no
setSelected(boolean)    //establece el estado (activado o
                        desactivado)
```

Emite un evento *ItemStateChanged* al cambiar de estado.

```
ItemEvent.SELECTED      // activado
ItemEvent.DESELECTED    // desactivado
```


6.3.4 ButtonGroup

Si se quieren utilizar varios botones *JRadioButton* (botones radio) de modo que solamente pueda haber uno seleccionado hay que crear un *ButtonGroup* que contenga dichos *JRadioButton*. Técnicamente, *Swing* permite que cualquier tipo de *AbstractButton* pueda ser añadido a un *ButtonGroup*, aunque lo habitual es utilizarlo con botones *RadioButton*.

Para crear un grupo de botones seleccionaremos del panel *Palette* el componente *ButtonGroup* y lo añadimos a la ventana. Luego, desde el panel de propiedades de cada botón, podemos seleccionar el grupo al que pertenecen.

Constructores:

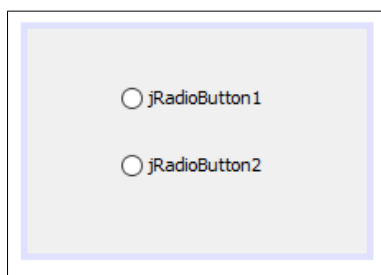
```
ButtonGroup()
```

Métodos

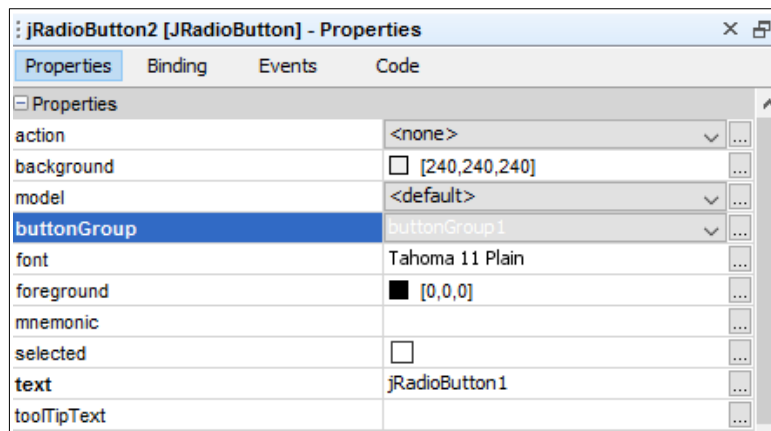
```
void add(AbstractButton)      //añade un botón al grupo
void remove(AbstractButton)   //quita un botón del grupo
Enumeration<AbstractButton> getElements() // devuelve los botones
int getButtonCount()          //devuelve el nº de botones
boolean isSelected(ButtonModel) //dice si un botón está activado
void setSelected(ButtonModel, boolean) // establece el estado del
botón
void clearSelection()          // desactiva todos los botones
```

Veamos un ejemplo paso a paso:

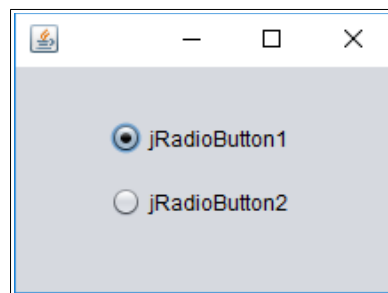
1. Insertamos un *ButtonGroup* en nuestro panel (téngase en cuenta que no es visible).
2. Introducimos en el panel dos *JRadioButton*.



3. Seleccionamos los botones y en *Properties* indicamos que pertenecen al grupo creado anteriormente.



4. Si lo ejecutamos, veremos que sólo uno de ellos podrá permanecer seleccionado a la vez:



6.4 Campos de texto

Swing introduce varios componentes que **permiten al usuario introducir y editar texto**. Los tres más importantes son:

```
JTextField  
JPasswordField  
JTextArea
```

Como de costumbre, estos componentes están disponibles en el *Palette* de NetBeans y desde ahí podemos añadirlos a nuestra aplicación gráfica. Desde la pestaña *Properties* podemos modificar su aspecto: *font*, *editable*, *toolTipText*, *text*, *background*, *foreground* y *enabled*.

6.4.1 JTextField

Campo de texto editable en una sola línea. Es el utilizado en el primer ejemplo creado.

Constructores:

```
JTextField()  
JTextField(String) // texto inicial del campo de texto  
JTextField(String, int) // texto inicial y n° de columnas
```

Métodos:

```
String getText() // devuelve el texto  
void setText(String) // establece el texto  
int getColumns() // devuelve el n.º de columnas  
void setColumns(int) // establece el n.º de columnas  
boolean getEditable() // devuelve si es editable  
setEditable(boolean) // establece si es editable
```

6.4.2 JPasswordField

Campo de texto editable para contraseñas en una sola línea. No muestra el texto que se escribe.

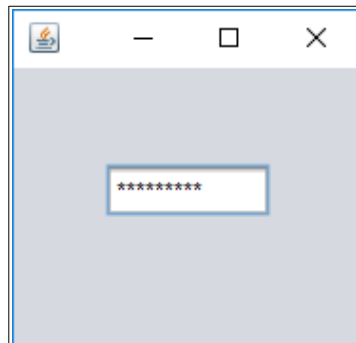
Constructores:

```
JPasswordField()  
JPasswordField(String)           // texto inicial  
JPasswordField(String, int)      // texto inicial y n.º de columnas
```

Métodos:

```
String getPassword()           // devuelve la contraseña  
void setPassword(String)       // establece la contraseña  
void setEchoChar(char)         // carácter a mostrar al escribir (por defecto *)  
void setColumns(int)           // establece el n.º de columnas
```

Su aspecto es el siguiente:



6.4.3 JTextArea

Campo de texto editable con varias filas y columnas.

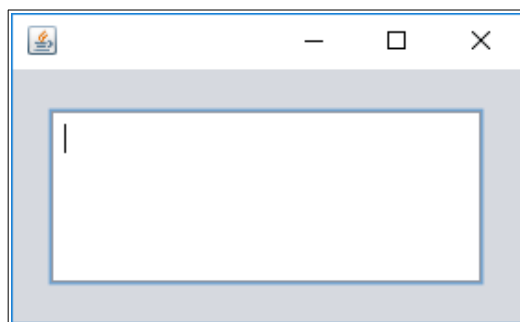
Constructores:

```
JTextArea()  
JTextArea(String)           // texto inicial  
JTextArea(int, int)         // n.º de filas y n.º de columnas  
JTextArea(String, int, int) // texto inicial, n.º de filas y n.º  
                             de columnas
```

Métodos

```
String getText()           // devuelve el texto  
void setText(String)       // establece el texto  
int getLineCount()         // devuelve el n.º de líneas  
String getSelectedText()   // devuelve el texto seleccionado  
void append(String)        // añade texto al final  
void insert(String, int)    // inserta texto en una posición  
void replaceRange(String, int, int) // sustituye texto en un rango
```

Su aspecto es el siguiente:

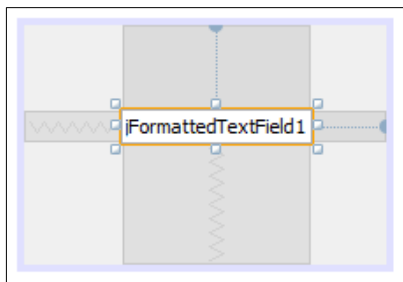


6.4.4 JFormattedTextField

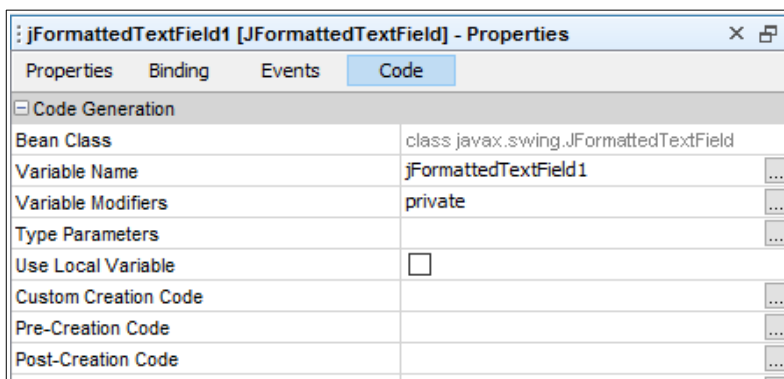
Campo de texto con formato. Puede configurarse para permitir un conjunto de caracteres que sigan un patrón determinado. Por ejemplo para introducir un DNI, fecha, número de teléfono, etc. El patrón se define mediante una máscara. Veamos un ejemplo.

Vamos a crear un campo de texto con formato y configurarlo con una máscara para DNI:

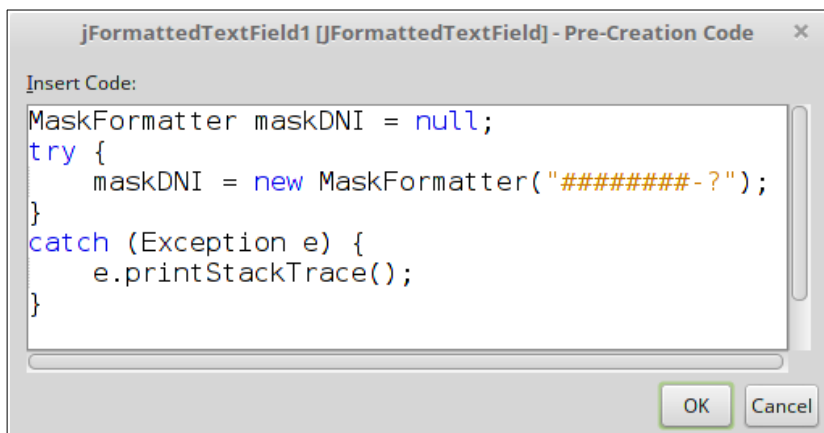
1. Seleccionamos el componente *JFormattedTextField* y lo insertamos en el panel.



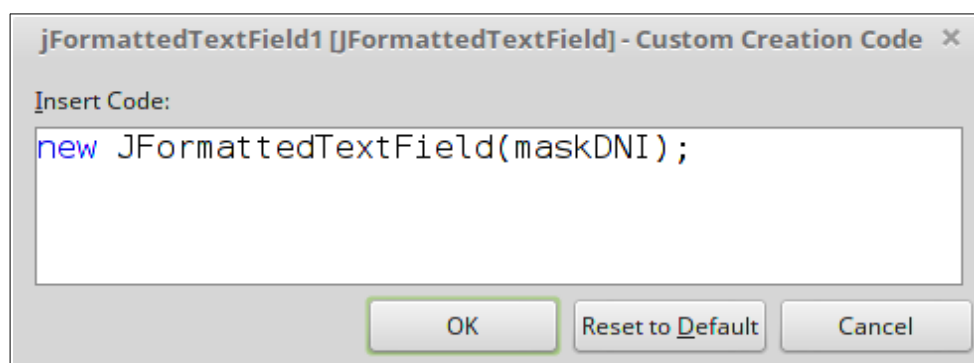
2. Junto a *Properties*, en la pestaña *Code*, hacemos click sobre el evento *Pre-Creation Code*. Esto nos permite definir código que se ejecutará antes de crear el campo de texto.



3. Introducimos el código de creación de un MaskFormatter que permitirá 8 números y 1 letra.



- Desde la pestaña *Code* del *JFormattedTextField* hacemos clic sobre el evento *Custom Creation Code*. Esto nos permite definir el código de creación del campo de texto. Introducimos el siguiente código. Como puede verse simplemente se instancia el objeto pasándole como argumento el *maskDNI* creado anteriormente.



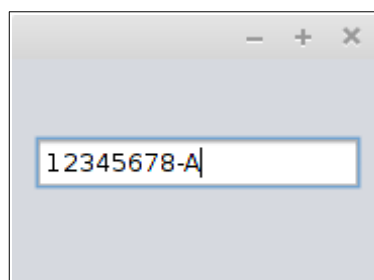
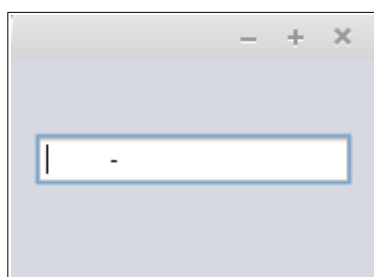
- Verás que NetBeans te avisa de un error de compilación. Necesitarás añadir al código del archivo un *import* de *javax.swing.text.MaskFormatter* y *javax.swing.JFormattedTextField*.

```
import javax.swing.JFormattedTextField;
import javax.swing.text.MaskFormatter;
```

Verás también que NetBeans ha añadido el código al archivo Java:

```
jButtonRes = new javax.swing.JButton();
jLabelRes = new javax.swing.JLabel();
MaskFormatter maskDNI = null;
try {
    maskDNI = new MaskFormatter("#####-?");
}
catch (Exception e) {
    e.printStackTrace();
}
jFormattedTextField1 = new JFormattedTextField(maskDNI);
setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
```

Al ejecutar la aplicación veremos que el campo de texto solo permite introducir 8 números y una letra, tal y como habíamos especificado con la máscara.



6.5 Rangos

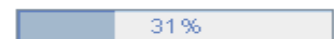
Un **rango** es una representación visual de información entre un mínimo y máximo. Es el caso de las barras de progreso y los sliders, que permiten mostrar información (barra de progreso) o introducir información (slider) de forma intuitiva para el usuario.

6.5.1 JProgressBar

Barra de progreso por ejemplo para mostrar el progreso de una instalación o el espacio ocupado en un disco duro.



Constructores:



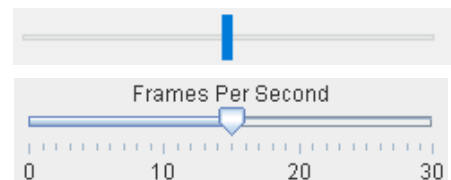
```
JProgressBar()           // barra horizontal entre 0 y 100
JProgressBar(int orientation) // JProgressBar.HORIZONTAL o
JProgressBar.VERTICAL
JProgressBar(int min, int max)
JProgressBar(int orientation, int min, int max)
```

Métodos:

```
int getValue()           void setValue(int)
int getMinimum()         void setMinimum(int)
int getMaximum()         void setMaximum(int)
double getPercentComplete() // devuelve progreso como porcentaje
```

6.5.2 JSlider

Slider por ejemplo para que el usuario controle fácilmente el volumen del sonido o el zoom de la aplicación.



Constructores:

```
JSlider()
JSlider(int orientation) //JSlider.HORIZONTAL o JSlider.VERTICAL
JSlider(int min, int max)
JSlider(int min, int max, int value)
JSlider(int orientation, int min, int max, int value)
```

Métodos:

```
int getValue()           void setValue(int)
int getOrientation()     void setOrientation(int)
int getMinimum()         void setMinimum(int)
int getMaximum()         void setMaximum(int)
```

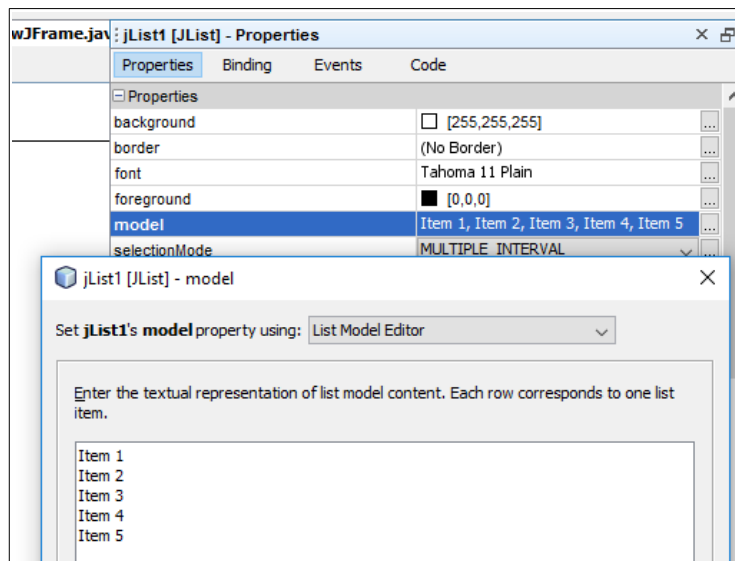

6.6 Listas y cajas

Permite al usuario seleccionar una opción o un conjunto de opciones de una lista.

6.6.1 JList

Lista de elementos. Pueden ser listas sencillas, de tamaño fijo, o variables, *ListModel*, que admiten diferentes modos de selección.

El nombre de los ítems se cambian desde la propiedad “*model*”.



Constructores:

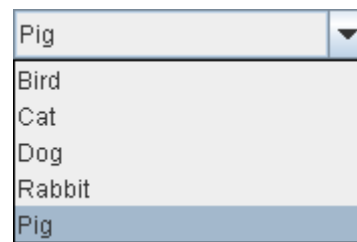
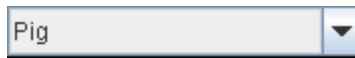
```
JList()
JList(Object[]) // lista a partir de un vector (String[])
JList(ListModel) // lista a partir de un ListModel
```

Métodos:

```
void setListData(Object[]) // establece la lista de elementos
void setSelectedIndex(int) // establece el elemento seleccionado
void setSelectedIndices(int[]) // establece los elementos seleccionados
int getSelectedIndex() // devuelve el índice del elemento seleccionado
int[] getSelectedIndices() // devuelve los índices de los elementos seleccionados
void setSelectionMode(int) // ListSelectionModel.SINGLE_SELECTION, SINGLE_INTERVAL_SELECTION o MULTIPLE_INTERVAL_SELECTION
int getSelectionMode() // devuelve el modo de selección
```

6.6.2 JComboBox

Combinación de entrada de texto con lista desplegable. El nombre de los ítems se cambia igual que en un *JList*.



Constructores:

```
JComboBox()  
JComboBox(Object[])           // lista a partir de un vector  
JComboBox(ComboBoxModel)     // lista a partir de un ComboBoxModel
```

Métodos:

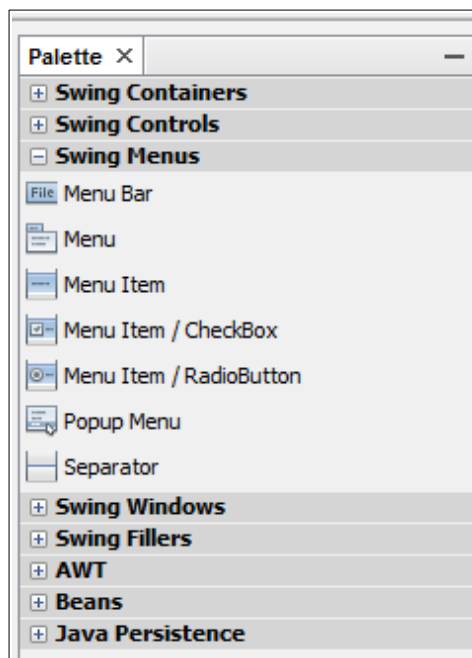
```
void addItem(Object)           //añade un item a la lista  
void insertItemAt(Object, int)  //inserta item en una posición  
Object getItemAt(int)          //devuelve el item en una posición  
Object getSelectedItem()       //devuelve el item seleccionado  
void removeAllItems()          //quita todos los items  
void removeItemAt(int)         //quita el item en una posición  
void removeItem(Object)        //quita el item pasado como argumento  
int getItemCount()             //devuelve cantidad de items
```

6.7 Menús JMenuBar, JMenu y JMenuItem

Los menús permiten crear listas de opciones y submenús con más opciones que el usuario puede utilizar para interactuar con la aplicación. Existen dos tipos principales:

- **Barra de menú:** menú estándar que con items y que puede contener otros menús.
- **Menú desplegable:** menú que se despliega cuando el usuario actúa sobre él.

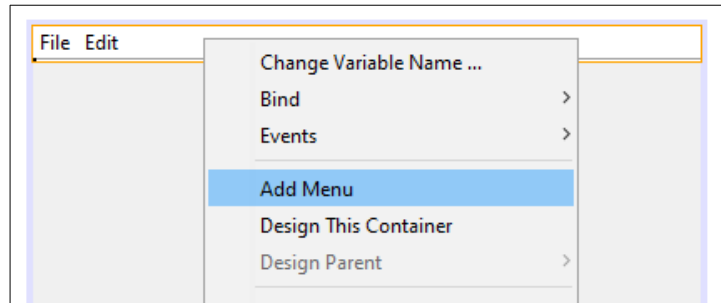
Para utilizar los menús deberemos acceder al apartado de *Menús* dentro de *Palette*:



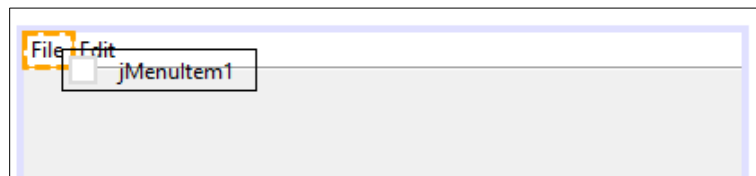
Veamos un ejemplo. Primero, tenemos que insertar en nuestra aplicación el componente *JMenuBar*. Como siempre pinchamos sobre él y lo arrastramos dentro del área de trabajo (*JFrame*).



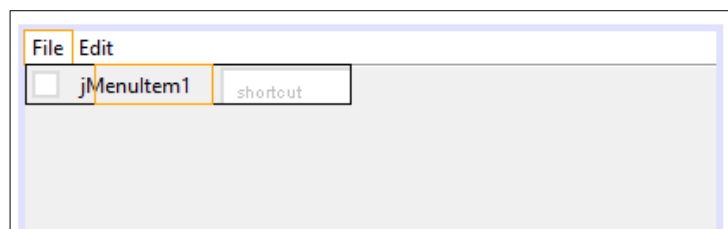
Si pinchamos sobre los menús creados (*File* o *Edit*) con el botón derecho podremos cambiarle el nombre del texto o de la variable. Podemos añadir más menús si pinchamos sobre la barra de menú con el botón derecho y luego en “*Add Menu*”.



A continuación, insertamos un componente *JMenu* dentro del menú *File*. Para ello lo arrastramos y lo soltamos encima de él.



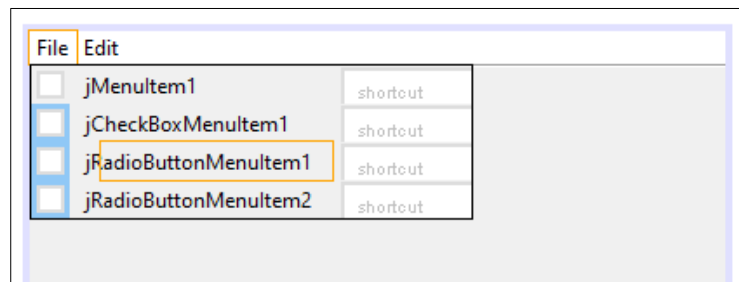
Una vez insertado, podemos cambiarle el nombre y especificarle el atajo de teclado.



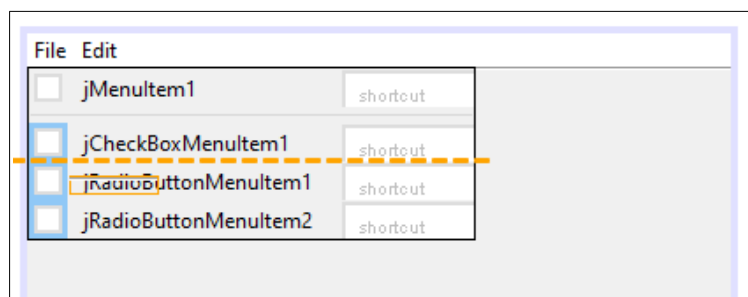
Hacemos lo mismo con el componente *JCheckBoxMenuItem* que contiene una casilla de verificación.



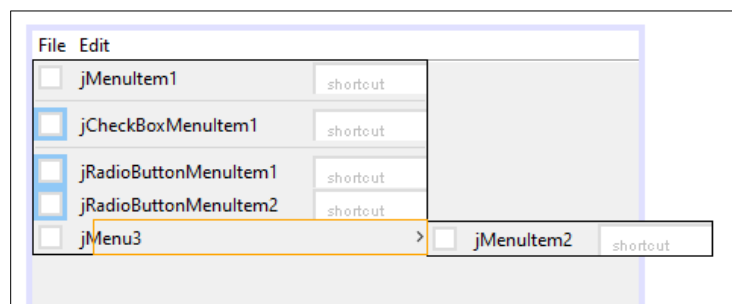
Y lo mismo con el componente *JRadioButtonMenuItem*. Recuerda que para que sólo haya uno seleccionado hay que asignarles el mismo *ButtonGroup* a todos.



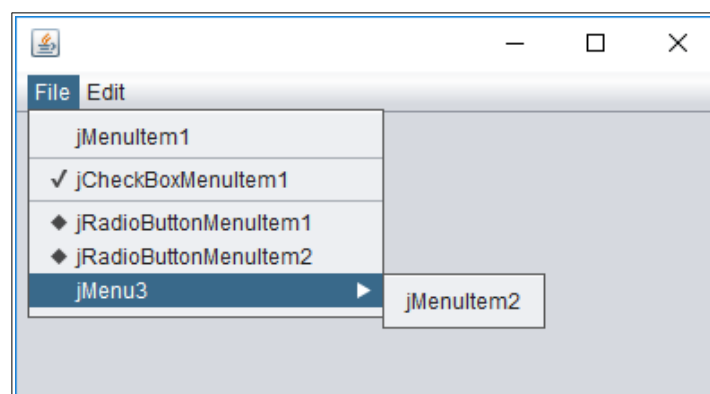
Podemos utilizar el componente *JSeparator* para separar con una línea cada elemento del menú.



Por último introducimos un componente *JMenu* y dentro de él un *JMenuItem*.

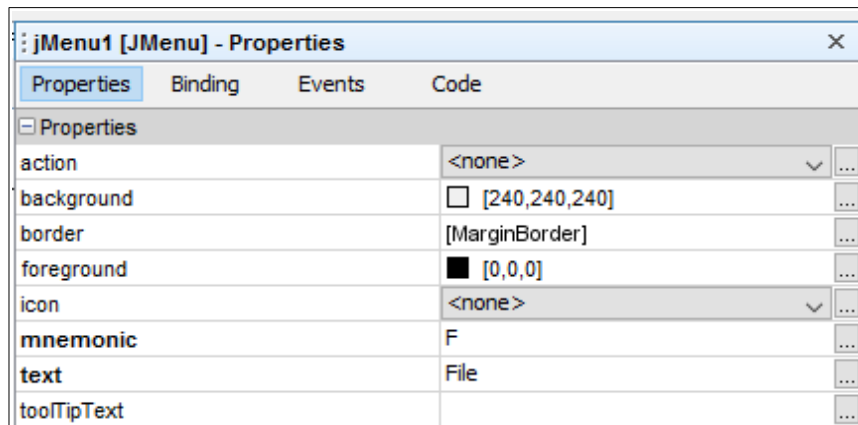


Siendo el resultado final este:

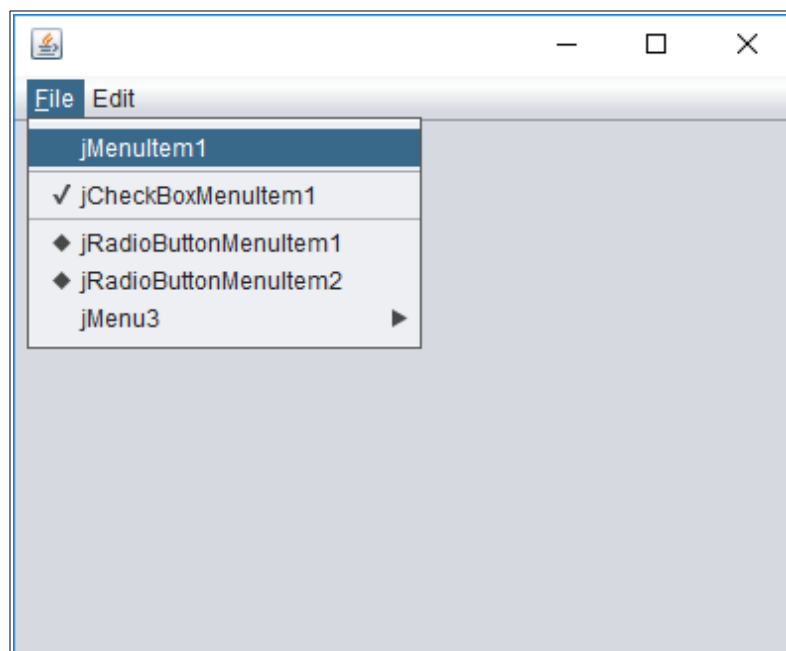


A continuación vamos a insertar los mnemónicos. Un **mnemónico** consiste en resaltar una tecla dentro de esa opción, mediante un subrayado, de forma que pulsando **Alt+<mnemónico del menú>** se abra el menú correspondiente, y volviendo a pulsar la letra correspondiente, **<mnemónico de la opción de menú>** se seleccione la acción correspondiente a esa opción del menú.

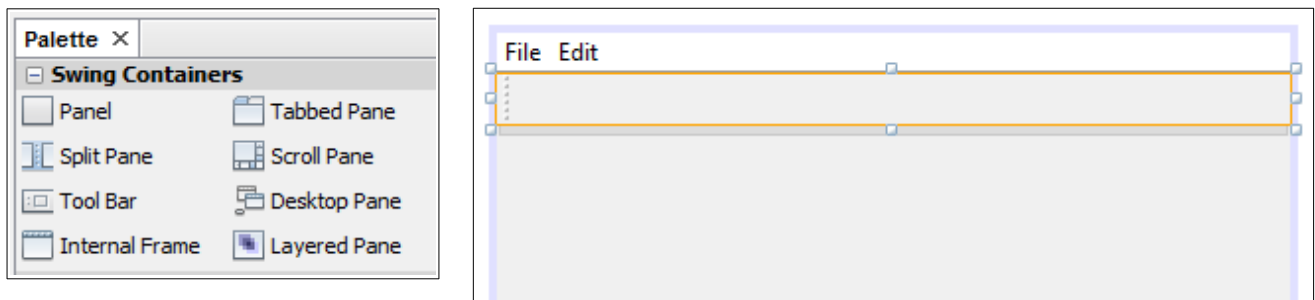
Para ello vamos a seleccionar el elemento *"File"* y en propiedades escribir una *F* en la opción *"mnemonic"*.



Así veremos que la *F* de *"File"* está subrayada y podremos abrir el menú pulsando *Alt+F*.



Por último vamos a insertar la típica barra de herramientas con iconos. Para ello seleccionamos y arrastramos el elemento *JToolBar* del grupo de contenedores (donde se encuentra los paneles).



Solo falta insertar botones dentro de la barra, para ello seleccionamos un botón y lo arrastramos dentro de la barra.

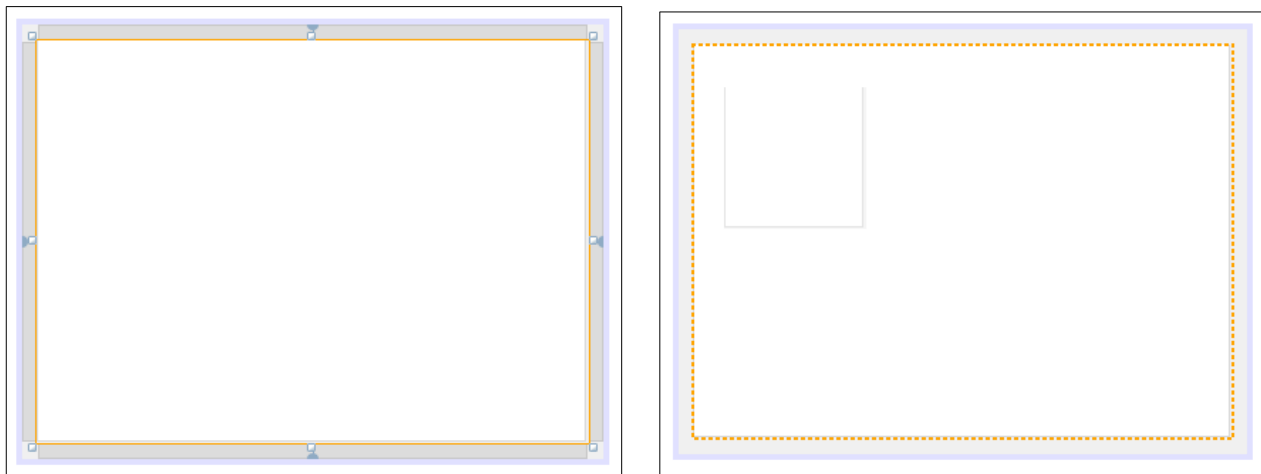


Solo faltaría asignarle un icono y programar los eventos.

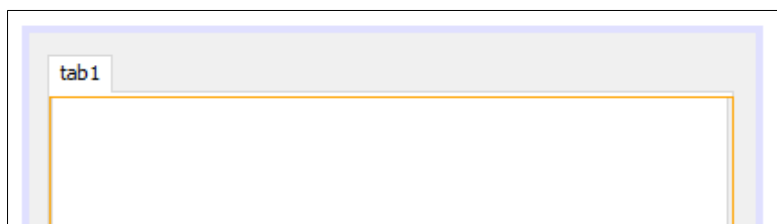
6.8 Panel con pestañas JTabbedPane

Un panel muy útil es el *JTabbedPane*, que es un panel con pestañas.

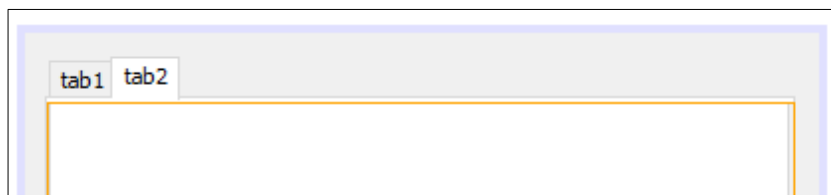
Para crear uno, lo primero es arrastrar el panel a la ventana y posicionarlo. A continuación, elegimos un *JPanel* y lo arrastramos, aunque con *JTabbedPane* también funcionaría.



Una vez hecho eso, nos aparecerá la primera pestaña.



Ahora podemos ir creando nuevas pestañas arrastrando nuevos *JPanel* o *JTabbedPane*.



Podemos configurar el nombre de cada *JTabbedPane* desde la pestaña de propiedades y añadir los elementos gráficos que queramos (contenedores, componentes, etc.) a cada uno de ellos como de costumbre.

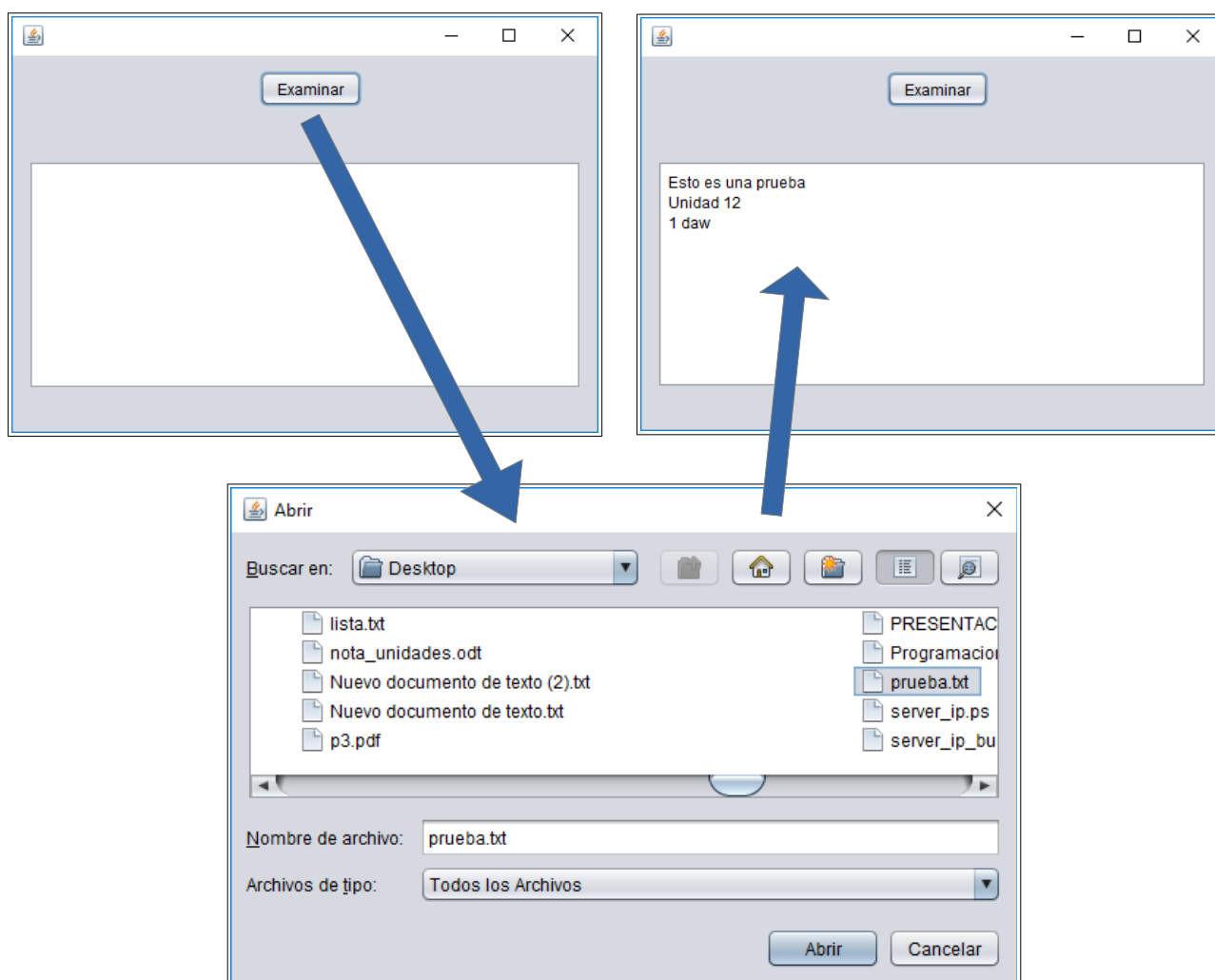
6.9 Selector de ficheros JFileChooser

En algunas aplicaciones se tiene la necesidad de leer o almacenar datos en fichero. Para ello es de utilidad el componente `JFileChooser` que nos permite mostrar al usuario una ventana emergente de selección de fichero.

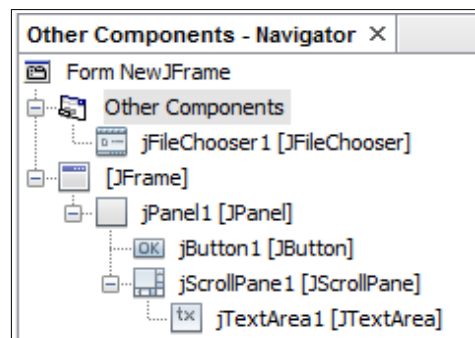
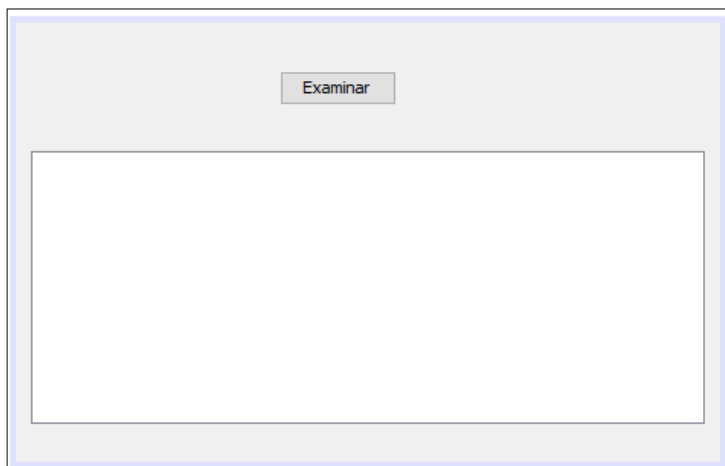
Para utilizarlo simplemente lo tendremos que seleccionar y arrastrar a la ventana de edición, no dentro del `JFrame`. Es un elemento que una vez insertado no es visible, aparecerá cuando lo activemos (al presionar un botón o un menú por ejemplo) pero sí aparece en el panel *Navigator*.

⚡ Ten cuidado, **no lo insertes dentro del `JFrame`**, ya que entonces sí aparecerá pero no funcionará correctamente.

Para ver su funcionamiento vamos a implementar una aplicación que pulsando un botón aparezca el selector (botón *Examinar*), seleccionamos un fichero y mostramos el contenido del fichero en un área de texto.



Veamos cómo hacerlo. Insertamos el selector de fichero, el panel, el botón y el área de texto.



A continuación, insertamos el código necesario en el evento del botón. Utilizaremos la clase *File* para guardar el fichero y la clase *BufferedReader*, pasándole el objeto *FileReader* con la ruta del fichero, para leer del mismo.

```

99 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
100     // TODO add your handling code here:
101     int seleccion = jFileChooser1.showOpenDialog(this); // mostramos el selector
102
103     if(seleccion == JFileChooser.APPROVE_OPTION) // Si no ha habido problemas
104     {
105         File fichero = jFileChooser1.getSelectedFile(); // Guardamos el fichero seleccionado en un
106
107         try
108         {
109             // Utilizamos la clase BufferedReader para leer el fichero
110             BufferedReader in = new BufferedReader(new FileReader(fichero.getAbsolutePath()));
111
112             String frase;
113
114             frase = in.readLine(); // Leemos una línea del fichero
115
116             while(frase != null) // Mientras leamos algo
117             {
118                 JTextArea1.setText(JTextArea1.getText() + frase + "\n"); // Actualizamos el área d
119                 frase = in.readLine();
120             }
121
122             in.close(); // Al finalizar cerramos la lectura del fichero.
123         }
124         catch(IOException e)
125         {
126             JOptionPane.showMessageDialog(this,"Error al leer el fichero.");
127         }
128     }
129 }
130

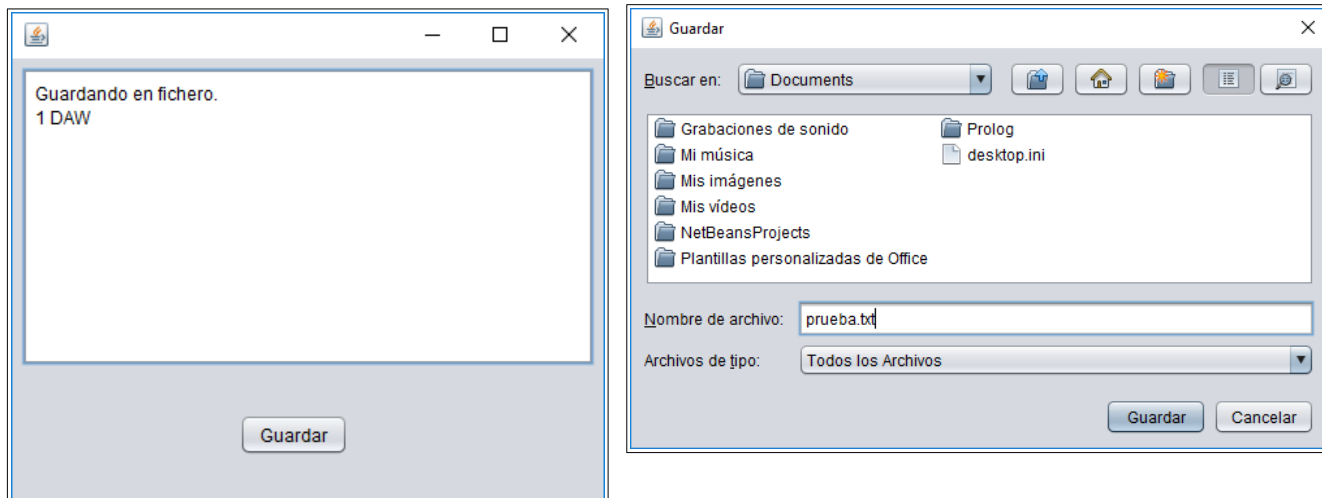
```

Cabe destacar que, en la línea 103, la variable selección puede ser igual a:

- `JFileChooser.CANCEL_OPTION` → Si el usuario le ha dado al botón cancelar.
- `JFileChooser.APPROVE_OPTION` → Si el usuario le ha dado al botón aceptar.
- `JFileChooser.ERROR_OPTION` → Si ha ocurrido algún error.

En la línea 123 vemos que tenemos que cerrar el fichero. No ha sido necesario abrirlo porque se hace automáticamente con el *FileReader*.

Ahora **veamos otro ejemplo** en el que guardamos el contenido de una caja de texto en un fichero. Para ello primero creamos una ventana parecida al ejemplo anterior.



A continuación insertamos el código necesario en el evento del botón. Utilizaremos la clase *File* para guardar el fichero y la clase *PrintWriter* para escribir en el fichero.

```
94 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {  
95     // TODO add your handling code here:  
96     int seleccion = jFileChooser1.showSaveDialog(this); // Mostramos el selector preparado para guardar  
97  
98     if(seleccion == JFileChooser.APPROVE_OPTION)  
99     {  
100         File fichero = jFileChooser1.getSelectedFile();  
101  
102         try  
103         {  
104             PrintWriter pw = new PrintWriter(fichero); // Preparamos el fichero para escribir en él  
105  
106             pw.print(jTextArea1.getText()); // Escribimos el contenido del área de texto  
107  
108             pw.close();  
109         }  
110         catch(IOException e)  
111         {  
112             JOptionPane.showMessageDialog(this, "Error al escribir en el fichero.");  
113         }  
114     }  
115 }  
116 }
```

6.10 Tablas: el componente JTable

Para crear una tabla la deberemos añadir como cualquier otro elemento al panel. Haciendo sobre ella clic derecho => “*Table Contents...*” podremos cambiar las propiedades del contenido de la tabla: filas, columnas, tipo de datos, si es editable o no, etc.

Title 1	Title 2	Title 3	Title 4

Customizer Dialog

Table Model Columns Rows

Title	Type	Resizable	Editable
Title 1	Object	☒	☒
Title 2	Object	☒	☒
Title 3	Object	☒	☒
Title 4	Object	☒	☒

Count: 4

Insert

Delete

Move Up

Move Down

Title: (No Property Editor) ☐ Resizable ☐ Editable

Type: Object Pref. Width: Default

Editor: (No Property Editor) Min. Width: Default

Renderer: (No Property Editor) Max. Width: Default

Selection Model: Not Allowed

☒ Allow to reorder columns by drag and drop

Close

Los datos de la tabla se almacenan en un objeto `TableModel` que pertenece al `JTable`. Para leer y modificar esos datos podemos obtener dicho objeto mediante el método `getModel()`:

```
TableModel getModel()    // devuelve el TableModel de la tabla
```

Algunos métodos útiles de la clase `TableModel` son:

```
int getRowCount()        //devuelve el nº de filas
int getColumnCount()     //devuelve el nº de columnas
Object getValueAt(int row, int col) //devuelve el valor de una celda
void setValueAt(Object value, int row, int col) //establece valor de celda
```

Sin embargo, la forma más sencilla de “alimentar” con datos un `JTable` es usando un objeto “`DefaultTableModel`”. Creamos el objeto, lo rellenamos y lo asignamos al `JTable`. Veamos un ejemplo sencillo:

```
DefaultTableModel model = new DefaultTableModel();

model.addColumn("ID");
model.addColumn("Nombre");
model.addColumn("Salario");

model.addRow(new Object[]{"1","Antonio García","1500€"});
model.addRow(new Object[]{"2","Ana Sánchez","2200€"});
model.addRow(new Object[]{"3","Manuel Pérez","1350€"});

tablaPersonas.setModel(model); //tablaPersonas es un JTable
```

Obviamente, podemos alimentar el objeto con cualquier fuente de datos, variables, etc...

7. CUADROS DE DIÁLOGO: JOPTIONPANE

Un cuadro de dialogo es una ventana que se abre desde una aplicación con el objetivo de interactuar con el usuario. Como su propio nombre indica, se utiliza para “dialogar” o intercambiar información entre la aplicación y el usuario. Para crear un diálogo, simple y estándar, se utiliza *JOptionPane*, que veremos a continuación. Para diálogos personalizados se utilizaría directamente la clase *Jdialog*. El código para diálogos simples debe ser mínimo y hay diferentes tipos.

Para todos ellos los diferentes tipos de opción a la hora de mostrar son:

- DEFAULT_OPTION
- YES_NO_OPTION
- YES_NO_CANCEL_OPTION
- OK_CANCEL_OPTION

Y los diferentes iconos:

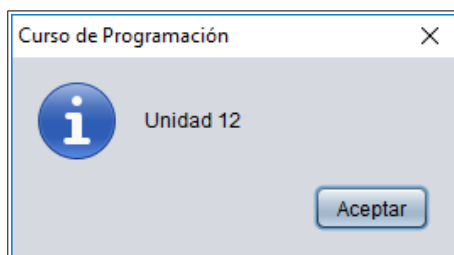
- ERROR_MESSAGE
- INFORMATION_MESSAGE
- WARNING_MESSAGE
- QUESTION_MESSAGE
- PLAIN_MESSAGE (Sin icono)

7.1 showMessageDialog

Muestra un diálogo modal con un botón etiquetado "OK" o "ACEPTAR". Se puede especificar fácilmente el mensaje, el título que mostrará el diálogo. Un ejemplo de código para mostrar este cuadro de dialogo es el siguiente:

```
JOptionPane.showMessageDialog(this, "Unidad 12", "Curso de  
Programación", JOptionPane.INFORMATION_MESSAGE);
```

Siendo el resultado:

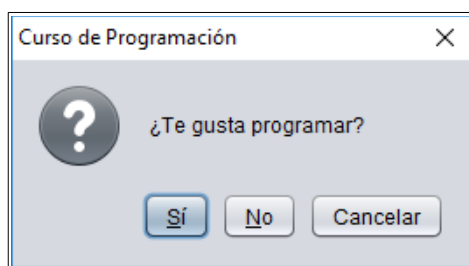


7.2 showConfirmDialog

Muestra un diálogo modal con tres botones, etiquetados con "Sí", "No" y "Cancelar". Devuelve un entero con la opción pulsada (0,1,2). Un ejemplo de código para mostrar este cuadro de dialogo es el siguiente:

```
int opcion = JOptionPane.showConfirmDialog(this, // Objeto actual
                                           "¿Te gusta programar?", // Texto del mensaje
                                           "Curso de Programación", // Título
                                           JOptionPane.YES_NO_CANCEL_OPTION); // Icono
```

Siendo el resultado:

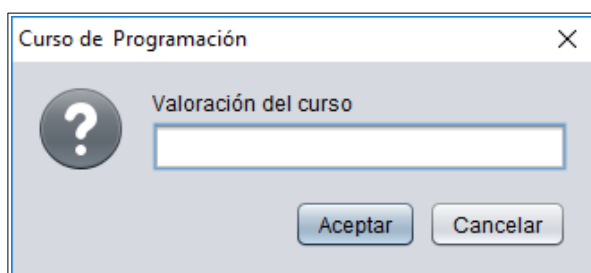


7.3 showInputDialog

Muestra un diálogo modal que obtiene una cadena del usuario (*String*). Un diálogo de entrada muestra un campo de texto para que el usuario teclee en él o un *ComboBox* no editable, desde el que el usuario puede elegir una de entre varias cadenas. Devuelve la opción elegida. Un ejemplo de código para mostrar este cuadro de dialogo con un *TextField* es el siguiente:

```
String valoracion = JOptionPane.showInputDialog(this, // Objeto actual
                                                "Valoración del curso", // Texto del mensaje
                                                "Curso de Programación", // Título
                                                JOptionPane.QUESTION_MESSAGE); // Icono
```

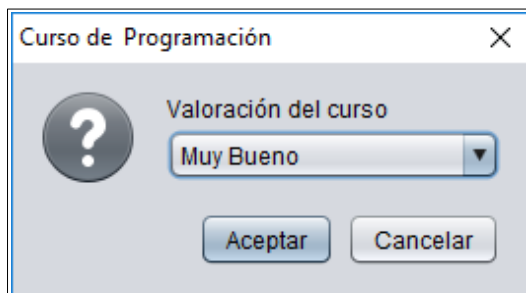
Siendo el resultado:



Un ejemplo de código para mostrar este cuadro de dialogo con el *ComboBox* sería el siguiente:

```
String[] opciones = {"Muy Bueno", "Bueno", "Malo", "Muy Malo"};  
String opc = (String) JOptionPane.showInputDialog(this, // Objeto actual  
    "Valoración del curso", // Texto del mensaje  
    "Curso de Programación", // Título  
    JOptionPane.QUESTION_MESSAGE, // Icono  
    null, // Parámetro no utilizado  
    opciones, // Vector de opciones  
    opciones[0]); // Valor a mostrar por defecto
```

Siendo el resultado:

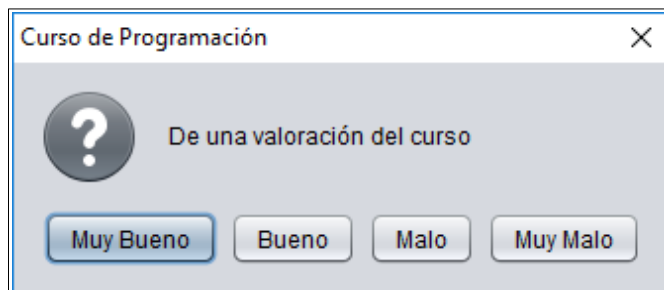


7.4 showOptionDialog

Muestra un diálogo modal con los botones, los iconos, el mensaje y el título especificado, etc. Con este método, podemos cambiar el texto que aparece en los botones de los diálogos estándar. También podemos realizar cualquier tipo de personalización. Un ejemplo de código para mostrar este cuadro de dialogo es el siguiente:

```
String[] opciones = {"Muy Bueno", "Bueno", "Malo", "Muy Malo"};  
int opc = JOptionPane.showOptionDialog(this, // Objeto actual  
    "De una valoración del curso", // Texto del mensaje  
    "Curso de Programación", // Título  
    JOptionPane.YES_NO_CANCEL_OPTION, // Opción  
    JOptionPane.QUESTION_MESSAGE, // Icono  
    null, // Parámetro no utilizado  
    opciones, // Vector de valores  
    opciones[0]); // Valor a mostrar por defecto
```

Siendo el resultado:



8. LAYOUTS: CONTROL DE DISTRIBUCIÓN DE COMPONENTES

Los Layouts permiten establecer reglas de diseño que determinan cómo se colocan y redimensionan los componentes dentro de un contenedor.

Tipos de Layouts

Swing proporciona varios tipos diferentes, cada uno diseñado para satisfacer diferentes necesidades de diseño y comportamientos. Algunos de los más comunes son:

a) **FlowLayout**: Este layout organiza los componentes en una fila o columna, añadiéndolos uno al lado del otro en la dirección especificada. Cuando la fila o columna se llena, los componentes se colocan en la siguiente fila o columna.

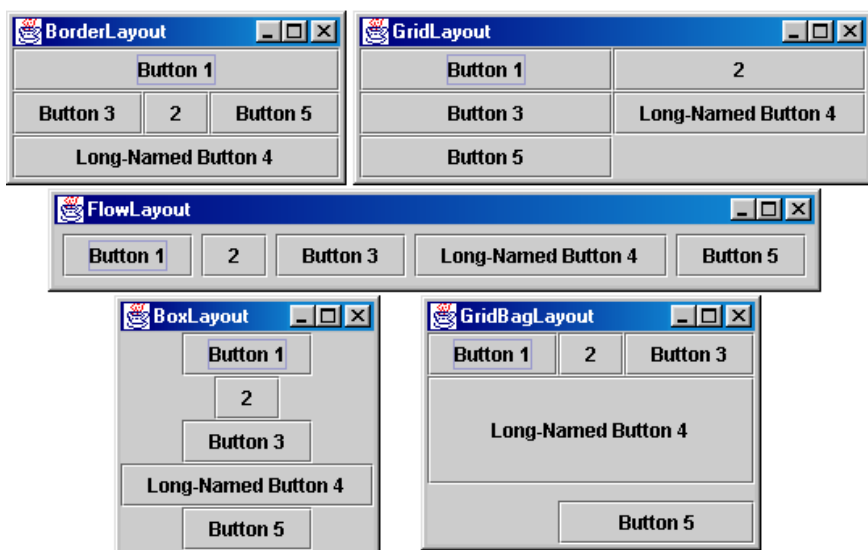
b) **BorderLayout**: Este layout divide el contenedor en cinco regiones: norte, sur, este, oeste y centro. Los componentes se colocan en una de estas regiones según las restricciones establecidas.

c) **GridLayout**: Este layout organiza los componentes en una cuadrícula de filas y columnas de igual tamaño. Todos los componentes tienen el mismo tamaño y se expanden o contraen para adaptarse al tamaño del contenedor.

d) **GridBagLayout**: Este layout proporciona un control más preciso sobre la disposición de los componentes. Permite especificar restricciones adicionales, como el ancho y alto mínimos y máximos, así como el espaciado entre componentes.

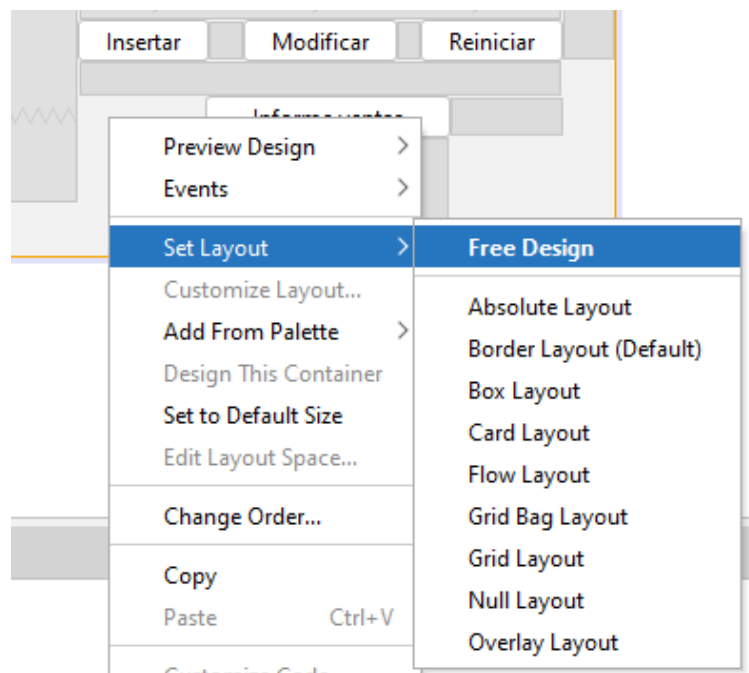
e) **BoxLayout**: Este layout organiza los componentes en una sola fila o columna. Es útil para diseños más simples donde se requiere una disposición lineal de los componentes.

Veamos un ejemplo de todos ellos:



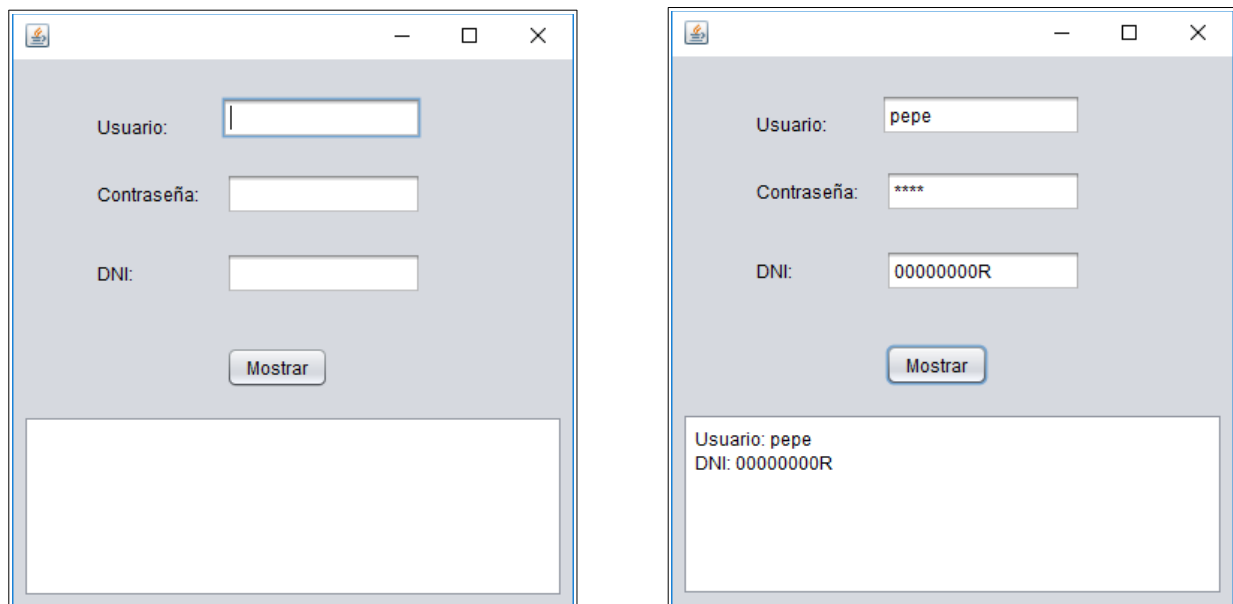
Por defecto, cada contenedor tiene un controlador de distribución -- un objeto que realiza el control de la distribución de los componentes dentro del contenedor. Los componentes pueden proporcionarle al controlador de disposición sus preferencias en cuanto a tamaño y alineamiento, pero la última palabra la tiene el controlador de disposición.

En el entorno de NetBeans, para poder especificar el layout a emplear lo haremos pulsando con el botón derecho del ratón sobre el contenedor a configurar. Se puede hacer directamente en el editor visual, o bien en el “Navigator”:



9. EJEMPLO GUIADO DE INTERFAZ SIMPLE

Realicemos un ejemplo completo donde veamos algunos de los elementos estudiados en la unidad. Para ello vamos a crear la siguiente aplicación, donde al pulsar en el botón “Mostrar” se muestre en el *textarea* el *usuario* y el *dni* introducido:

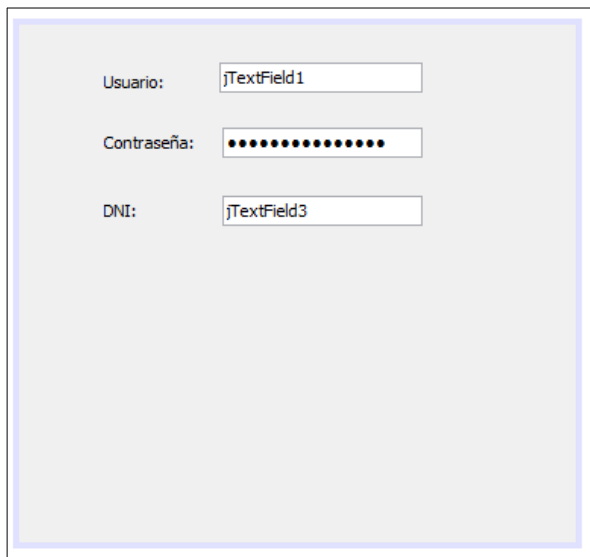


Para ello, primero crearemos un proyecto recordando desmarcar la opción “*Create Main Class*”. Y seguiremos los siguientes pasos:

1. Insertar y posicionar el panel y las etiquetas (cambiamos el nombre pulsando dos veces sobre ella o pulsando en botón derecho sobre ella y seleccionando “*Edit Text*”):



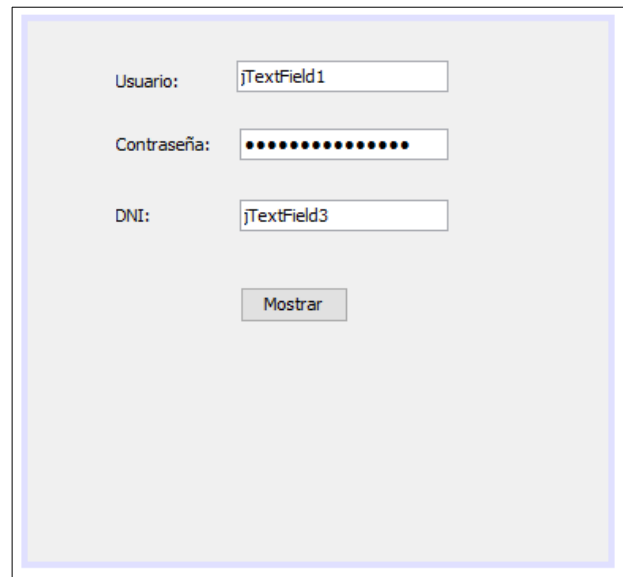
2. Insertar y posicionar los *textfields*. Luego insertamos y posicionamos el botón (cambiamos el nombre pulsando en botón derecho sobre él y seleccionando “Edit Text”):



Usuario:

Contraseña:

DNI:

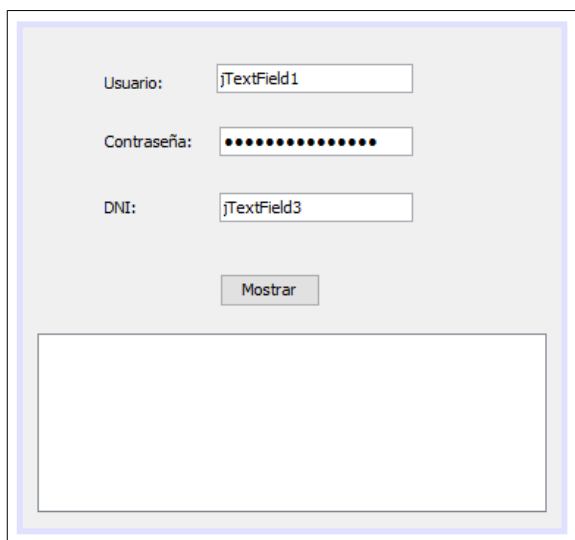


Usuario:

Contraseña:

DNI:

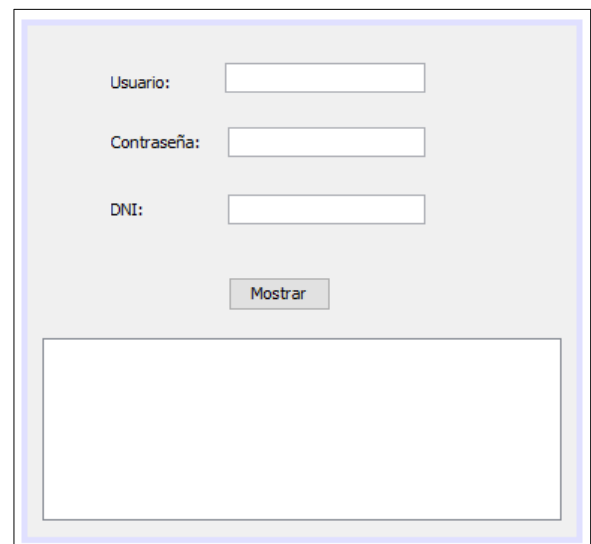
3. Insertamos y posicionamos el *textarea*. Eliminamos el texto de los *textfields* de la misma forma que cambiamos el nombre a la etiqueta o al botón:



Usuario:

Contraseña:

DNI:



Usuario:

Contraseña:

DNI:

4. Pinchamos dos veces sobre el botón y creamos el evento *actionPerformed*:

```
122 |  
123 | private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {  
124 |     // TODO add your handling code here:  
125 | }  
126 |
```

5. Insertamos en el método el código necesario para insertar en el *textarea* los datos. Para recoger el texto introducido en los campos de texto utilizaremos el método *getText()* y para insertarlo en el área de texto utilizaremos el método *append(String)*.

```
129 |  
130 | private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {  
131 |     // TODO add your handling code here:  
132 | jTextArea1.append("Usuario: " + jTextField1.getText() + "\n");  
133 | jTextArea1.append("DNI: " + jTextField3.getText() + "\n");  
134 | }
```

10. PATRONES DE DISEÑO Y ARQUITECTURAS SOFTWARE

Los **patrones de diseño** son técnicas que permiten resolver problemas comunes en el desarrollo de software y en otros ámbitos referentes al diseño de interacción o interfaces. Un patrón de diseño es como una plantilla para tu proyecto; usa ciertas convenciones y puedes esperar un comportamiento de tipo específico. No son propios de Java, sino que son un concepto transversal, sin embargo los vamos a introducir ahora dada su importancia a medida que las aplicaciones que diseñamos crecen en tamaño y complejidad. Y por eso mismo son necesarios, ya que nos permiten organizar y estructurar las aplicaciones de forma que éstas sean manejables y mantenibles.

Los patrones de diseño pretenden:

- Proporcionar elementos reutilizables en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y resueltos.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.

Por tanto no es obligatorio utilizar los patrones, sólo es aconsejable en el caso de detectar el mismo problema o uno similar para el cual existe un patrón que proporciona una solución probada, siempre teniendo en cuenta que en un caso particular puede no ser aplicable. Abusar o forzar el uso de los patrones puede ser contraproducente.

Existen muchos patrones de diseño, pero nosotros destacaremos los siguientes:

- Factory
- Singleton
- Observer
- Strategy
- Decorator

Respecto a las **arquitecturas de software**, éstas se refieren a la estructura fundamental de un proyecto software, incluyendo sus componentes, sus relaciones y las reglas y directrices que gobiernan su organización.

Estas arquitecturas proporcionan una visión global del sistema y establecen la base sobre la cual se construye y evoluciona el software. Los proyectos de desarrollo, a medida que crecen, requieren metodologías de gestión; para ello, las arquitecturas de software ofrecen muchas ventajas, como: organización y estructura, escalabilidad, flexibilidad, mantenibilidad y reutilización de código.

Una arquitectura de software sólida es fundamental para el éxito a largo plazo de un proyecto de desarrollo de software.

Veamos algunas de las arquitecturas software más destacadas:

- MVC (Modelo-Vista-Controlador).
- MVP (Modelo-Vista-Presentador).
- MVVM (Modelo-Vista-VistaModelo).
- Microservicios.
- Monolítica.

Por el momento nos centraremos únicamente en el MVC, una de las arquitecturas más destacadas y semilla de muchas otras (como MVP y MVVM).

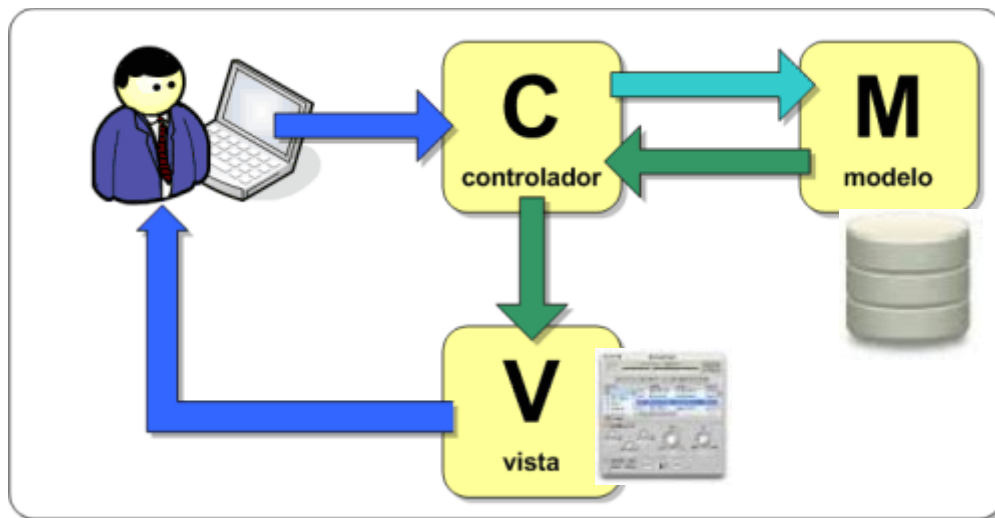
10.1 MVC (Modelo, Vista, Controlador)

MVC, que son las siglas de “Modelo, Vista, Controlador”, es un patrón de diseño que organiza la lógica de una aplicación en distintas capas (concretamente 3), cada una de las cuales lleva a cabo un conjunto específico de tareas. Las capas interactúan entre sí para garantizar que la funcionalidad de la aplicación se entregue de manera coordinada y consistente. La metodología MVC involucra a toda la aplicación, desde la interfaz de usuario (UI) hasta el modelo de datos subyacente.

MVC ofrece importantes ventajas a los equipos de desarrollo. Los programadores pueden construir componentes simultáneamente sin interferir en el trabajo de los demás, y pueden reutilizarlos. MVC facilita la creación de aplicaciones grandes y complejas, lo que genera desarrollos más rápidos y eficientes. Además, MVC admite el desarrollo basado en pruebas (TDD), al tiempo que permite probar y solucionar problemas de componentes individualmente.

La metodología MVC separa la lógica de una aplicación en 3 capas distintas:

1. **Modelo.** Responsable de la lógica de datos de la aplicación, y de consultas bases de datos u otros dispositivos de almacenamiento. Esta capa podría incluir mecanismos de validación y realizar otras tareas relacionadas con los datos. Además, es responsable de mantener todos los aspectos de los datos y garantizar su integridad y accesibilidad.
2. **Vista.** Esta capa proporciona la interfaz de usuario necesaria para interactuar con la aplicación. Incluye los componentes para mostrar los datos y permitir a los usuarios interactuar con ellos. Por ejemplo, la capa de vista puede incluir formularios, botones, enlaces, tablas, listas desplegadas o cuadros de texto, entre otras cosas.
3. **Controlador.** Contiene la lógica de aplicación necesaria para facilitar las comunicaciones entre la aplicación, actuando como una interfaz entre las capas de la vista y el modelo. A veces, se considera que el controlador es el cerebro de la aplicación, que mantiene todo en movimiento y sincronizado.



Veamos un ejemplo sencillo de una calculadora con interfaz gráfica basada en el MVC:

1. **Modelo:** contendrá los métodos para realizar las operaciones matemáticas.
2. **Vista:** contendrá los componentes de la interfaz gráfica de la calculadora.
3. **Controlador:** manejará las interacciones del usuario y llamará a los métodos correspondientes en el modelo.

```
// Modelo
public class CalculadoraModelo {
    public int sumar(int num1, int num2) {
        return num1 + num2;
    }

    public int restar(int num1, int num2) {
        return num1 - num2;
    }

    public int multiplicar(int num1, int num2) {
        return num1 * num2;
    }

    public int dividir(int num1, int num2) {
        if (num2 == 0) {
            throw new IllegalArgumentException("No se puede dividir por cero!");
        }
        return num1 / num2;
    }
}
```



```
// Vista
import javax.swing.*;

public class CalculadoraVista {
    private JTextField firstNumber = new JTextField(10);
    private JLabel additionLabel = new JLabel("+");
    private JTextField secondNumber = new JTextField(10);
    private JButton calculateButton = new JButton("Calcular");
    private JTextField result = new JTextField(10);

    public CalculadoraVista() {
        JPanel calcPanel = new JPanel();

        calcPanel.add(firstNumber);
        calcPanel.add(additionLabel);
        calcPanel.add(secondNumber);
        calcPanel.add(calculateButton);
        calcPanel.add(result);

        JFrame frame = new JFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.getContentPane().add(calcPanel);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }

    public int getFirstNumber() {
        return Integer.parseInt(firstNumber.getText());
    }

    public int getSecondNumber() {
        return Integer.parseInt(secondNumber.getText());
    }

    public void setResult(int resultValue) {
        result.setText(Integer.toString(resultValue));
    }

    public void displayErrorMessage(String errorMessage) {
        JOptionPane.showMessageDialog(null, errorMessage);
    }

    public void addCalculateListener(ActionListener listener) {
        calculateButton.addActionListener(listener);
    }
}
```

```
// Controlador
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class CalcControlador {
    private CalculadoraVista view;
    private CalculadoraModelo model;

    public CalcControlador(CalculadoraVista view, CalculadoraModelo model) {
        this.view = view;
        this.model = model;

        this.view.addCalculateListener(new CalculateListener());
    }

    class CalculateListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            try {
                int num1 = view.getFirstNumber();
                int num2 = view.getSecondNumber();

                int sum = model.add(num1, num2);
                view.setResult(sum);
            } catch (NumberFormatException ex) {
                view.displayErrorMessage("Escriba números válidos.");
            }
        }
    }

    public static void main(String[] args) {
        CalculadoraVista view = new CalculadoraVista();
        CalculadoraModelo model = new CalculadoraModelo();
        CalcControlador controller = new CalcControlador(view, model);
    }
}
```

10.2 Singleton

Se trata de un patrón de diseño creacional que garantiza que una clase tenga solo una instancia y proporciona un punto de acceso global a esa instancia. Esto es útil cuando solo se necesita una única instancia de una clase para coordinar acciones en todo el sistema, como por ejemplo para representar una conexión a base de datos desde la aplicación. En la siguiente unidad, en el apartado de acceso a base de datos, detallaremos con un ejemplo el funcionamiento de este patrón y su aplicación más utilizada.

10.3 Observer

Es un patrón de diseño de comportamiento que define una relación de uno a muchos entre objetos, de modo que cuando un objeto cambia de estado, todos los objetos que dependen de él son notificados y actualizados automáticamente.

Este patrón se utiliza comúnmente en la implementación de sistemas donde un objeto necesita notificar a múltiples objetos sobre cambios en su estado, como en el diseño Modelo-Vista-Controlador (MVC) o en aplicaciones que requieren manejo de eventos.

Veamos un ejemplo utilizando el patrón Observer en un contexto real. Supongamos que estamos desarrollando una aplicación de clima donde queremos mostrar las actualizaciones del clima en múltiples dispositivos de usuario en tiempo real. Cada dispositivo de usuario actuará como un observador que recibirá actualizaciones del clima cuando se produzcan cambios en los datos meteorológicos.

El proveedor de datos meteorológicos notificará a los dispositivos de usuario registrados sobre las actualizaciones del clima, y cada dispositivo de usuario reaccionará a las actualizaciones de manera específica.

```
import java.util.ArrayList;
import java.util.List;

interface ObservadorClima {
    void actualizar(String datosClima);
}

class Dispositivo implements ObservadorClima {
    private String nomDisp;

    public Dispositivo(String nomDisp) {
        this.nomDisp = nomDisp;
    }

    @Override
    public void actualizar(String datosClima) {
        System.out.println(nomDisp + " recibió actualización: " + datosClima);
    }
}

class ProveedorClima {
    private List<ObservadorClima> observadores = new ArrayList<>();

    public void agregarObservador(ObservadorClima observador) {
        observadores.add(observador);
    }

    public void delObservador(ObservadorClima observador) {
        observadores.remove(observador);
    }
}
```

```
}

public void notificar(String datosClima) {
    for (ObservadorClima observador : observadores) {
        observador.actualizar(datosClima);
    }
}

public void obtenerDatosNotificar() {
    String datosClima = "Soleado, 25°C";
    notificar(datosClima);
}
}

public class AppClima {
    public static void main(String[] args) {
        ProveedorClima proveedor = new ProveedorClima();

        Dispositivo telefono = new Dispositivo("Teléfono");
        Dispositivo tablet = new Dispositivo("Tableta");
        Dispositivo portatil = new Dispositivo("Portátil");
        proveedor.agregarObservador(telefono);
        proveedor.agregarObservador(tablet);
        proveedor.agregarObservador(portatil);

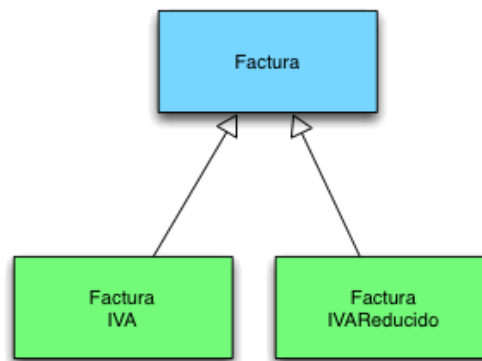
        proveedor.obtenerDatosNotificar();
    }
}
```

10.4 Factory

Es un patrón de diseño creacional que se utiliza para encapsular la creación de objetos. Su objetivo principal es proporcionar una interfaz para crear objetos de un tipo determinado sin especificar explícitamente la clase concreta de objeto que se creará. Esto permite desacoplar el código cliente del código de creación de objetos y proporciona flexibilidad en la creación de objetos en tiempo de ejecución.

El patrón Factory generalmente consiste en una interfaz o clase base común (a menudo llamada fábrica) con un método o métodos que son responsables de crear objetos. Luego, hay clases concretas de fábrica que implementan este método para crear instancias específicas de objetos.

Un ejemplo concreto de uso del patrón Factory podría ser en un sistema de gestión de vehículos donde se necesitan crear diferentes tipos de vehículos, como automóviles, motocicletas y camiones. En lugar de que el código cliente tenga que saber cómo crear cada tipo de vehículo, se utiliza una fábrica para ocultar esta lógica de creación. Veamos también un ejemplo diferente basado en una jerarquía de facturas:



```
public abstract class Factura {
    private int id;
    private double importe;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public double getImporte() {
        return importe;
    }
    public void setImporte(double importe) {
        this.importe = importe;
    }
    public abstract double getImporteIva();
}

public class FacturaIvaReducido extends Factura{
    @Override
    public double getImporteIva() {
        return getImporte()*1.07;
    }
}

public class FacturaIva extends Factura {
    @Override
    public double getImporteIva() {
        return getImporte() * 1.21;
    }
}

// Factoría para construir ambos objetos de la jerarquía
public class FactoriaFacturas {
    public static Factura getFactura(String tipo) {
        if (tipo.equals("iva")) {
            return new FacturaIva();
        }
    }
}
```

```
        } else {  
            return new FacturaIvaReducido();  
        }  
    }  
}  
  
public class Principal {  
    public static void main(String[] args) {  
        Factura f = FactoriaFacturas.getFactura("iva");  
        f.setId(1);  
        f.setImporte(100);  
        System.out.println(f.getImporteIva());  
    }  
}
```

Si nos fijamos, la clase `FactoriaFacturas` lo único que hace es instanciar un objeto u otro dependiendo del tipo que le solicitemos. Eso en un principio parece poco práctico. Pero fijémonos en cómo queda el programa “main”. Nos daremos cuenta que el programador únicamente tiene que tratar con el concepto de `Factura` para él, las clases `FacturaIva` y `FacturaReducido` no existen.

Esto permite una simplificación a la hora de trabajar. Es cierto que las `Factorias` se encargan de generar una jerarquía de clases pero su función fundamental es encapsular una jerarquía de objetos y reducir el conjunto de conceptos con los que trabajamos.